



Dipartimento di Ingegneria e Architettura

CORSO DI LAUREA IN INGEGNERIA DELL'ENERGIA
ELETTRICA E DEI SISTEMI

PROVA FINALE

AN IMPLEMENTATION OF OPEN-RMF IN AN INDUSTRIAL ENVIRONMENT

Laureando: Michele Belletti

Relatore: Stefano Seriani
Correlatori: Roberto Pugliese
Georgios Kourousias

ANNO ACCADEMICO 2024-2025

Contents

I	Introduction	5
1	Background and motivation	7
2	Research questions and objectives	8
3	Scope and limitations	10
II	Literature Review	11
4	Overview of ROS1 and ROS2	13
4.1	Structure of ROS	15
4.2	Tools of ROS	17
5	Introduction to NAV2 and Google Cartographer	19
5.1	Description of Nav2	19
5.1.1	Cartographer	23
6	Review of related work on porting navigation stack from ROS 1 to ROS 2	24
7	Overview of open source fleet management systems, particularly Open-RMF	25
7.1	Open-RMF description	26
III	Methodology	33
8	Description of research platform and hardware used	35
9	Steps taken to port the navigation stack from ROS 1 to ROS 2	39
9.1	Odometry Estimation odom → base_link	39
9.1.1	Configuration Odometry Estimation in ekf_node	39
9.2	Localization map → odom	40
9.2.1	Mapping	40
9.2.2	Only localization	41
9.3	Structure Navigation Stack Nav2	42
9.3.1	Navigation Launch file	42
9.3.2	Porting ROS 1 DStarGlobalPlanner to ROS 2	44

10 Jobot porting procedure to ROS 2	47
10.1 Jobot Description	47
10.2 What changes where made in the code	47
10.3 Create a model for the simulation	49
11 Integration of the navigation stack with Open-RMF	50
11.1 Traffic Management - Route Maps Generation	50
11.2 RMF core	53
11.3 Free Fleet	54
11.3.1 Setup fleet	54
11.3.2 Setup Zenoh	55
11.3.3 Launching Free Fleet	55
11.4 Web interfaces	56
12 Testing and evaluation of the system	57
12.1 Testing environment using Rosbag	57
12.2 Parameters setting for the robot_localization	59
12.2.1 Setting up the EKF of the mini	61
12.2.2 Setting up the EKF of the Jobot	65
12.3 Parameter tuning for the AMCL	68
12.3.1 Setting up the AMCL of the mini	68
12.3.2 Setting up the AMCL for the Jobot	72
12.4 Navigation problems	74
12.4.1 In simulation	74
12.4.2 In reality	74
12.5 Other problems discovered during testing	76
IV Results and Analysis	80
13 Presentation of results from testing and evaluation	81
13.1 Porting the planner	82
13.2 Build 2 AMR in ROS 2	83
13.2.1 In particular for the Mini	83
13.3 In particular for the jobot	84
13.4 Open-RMF navigation result	84
14 Comparative analysis on the port system	86
14.1 Planner calculation time comparison	86
V Conclusion and Future Work	89
15 Conclusion	91
16 Future work	92

VI	Appendices	93
17	Simulation	94

Part I

Introduction

This thesis focuses on several key areas in robotics, particularly exploring the requirements for constructing an Autonomous Mobile Robot (AMR) from an Unmanned Ground Vehicle (UGV) using Nav2. Additionally, it examines the process of porting a global planner from ROS 1 to ROS 2. The work also includes the migration of an older ROS 1 AMR to ROS 2 and the integration of these systems into a new fleet management framework, OPEN-RMF. While OPEN-RMF is still under active development, it has reached a sufficiently mature state to effectively control mobile robots at Elettra Sincrotrone Trieste.

Chapter 1

Background and motivation

Elettra Sincrotrone Trieste¹ is a major research facility that primarily provides users access to two light sources: the synchrotron Elettra and the free electron laser FERMI. In addition to operating these sources, the facility actively researches new technologies and techniques to enhance its capabilities and operations. Many recent advancements are based on robotics, which, especially after the COVID-19 pandemic and with the rapid rise of large language models (LLMs), have enabled new ways for users to interact with the infrastructure, including robot-assisted information retrieval and package delivery services. Previously, the facility developed a global planner, D-Star (D-star based optimized trajectory planner)[19], within the ROS 1 framework to enable robot navigation across large-scale environments. However, as ROS 1 has reached end-of-life, the codebase must now be migrated to ROS 2.

Additionally, the facility has multiple robots, some of which are yet to be built, while others currently operating on ROS 1 that need to be ported also to ROS 2. Furthermore, the facility utilizes robots from various vendors, each specialized for different tasks and not designed for seamless coordination with one another and all is in an environment that has narrow passages and lifts can create significant coordination challenges during navigation. To address this issue, Open-RMF² has been developed in recent years as a multiplatform system for controlling and managing heterogeneous fleets of robots.

¹<https://www.elettra.eu/>

²<https://www.open-rmf.org/>

Chapter 2

Research questions and objectives

The objectives of this thesis are: to port the D-Star global planner from ROS 1 to ROS 2, to integrate a Rover Mini unmanned ground vehicle (UGV) from the company RoverRobotics ¹ into the ROS 2 Nav2 ecosystem making it a Autonomous Mobile Robot (AMR),



Figure 2.1: RoverRobotics Rover Mini

to port the D-Star global planner from ROS 1 to ROS 2, upgrade our Jobot AMR from ROS 1 to ROS 2,



Figure 2.2: Jobot Rover

incorporate the robot into the Open-RMF framework for centralized fleet manage-

¹<https://roverrobotics.com/>

ment and evaluate the suitability of the Open-RMF ecosystem for deployment at the Elettra Sincrotrone Trieste.

Chapter 3

Scope and limitations

The principal limitation is the current lack of access to lift control systems, which prevents direct integration and operation by Open-RMF. Additionally, there is no automated system for opening and closing doors, another feature that can be managed by Open-RMF. While Open-RMF offers a high degree of customizability, and human-in-the-loop interactions can be configured—such as triggering a light or sending a notification when a robot approaches a door—this method requires a human to manually open the door or operate the lift. However, such an approach is not ideal for achieving full autonomy.

Part II

Literature Review

In this section, we provide a description of the key technologies used in this thesis. We begin by describing the Robot Operating System (ROS) and its functionality, followed by an overview of the navigation system employed, and a discussion of how the Open-RMF fleet management system operates.

Chapter 4

Overview of ROS1 and ROS2

The Robotics Operating System (ROS)¹ is an open-source collection of tools and libraries designed to support the development of robotics applications[11]. Despite its name, ROS is not an operating system (OS) in the traditional sense, but rather a software development kit (SDK) that provides developers with a comprehensive set of tools for building robotic systems. It offers several features typically associated with an OS, including hardware abstraction, low-level device control, implementation of commonly used functionalities, inter-process communication and package management. The ROS ecosystem is built upon the following core components:

- **Middleware communication layer (RMW):** Built on the Data Distribution Service (DDS) standard, this layer enables nodes to exchange information using an anonymous publish/subscribe pattern. This design promotes fault isolation, separation of concerns and modular interfaces. ROS also supports synchronous, Remote Procedure Call (RPC)-style communication between services.

While the default DDS implementation is eProsima Fast DDS (`rmw_fastrtps_cpp`), this work uses Eclipse Cyclone DDS (`rmw_cyclonedds_cpp`) due to its better compatibility with the Nav2 navigation stack in ROS Humble.

- **Multiple Client Libraries:** Allow to program in ROS using multiple languages and access to the core communication APIs.

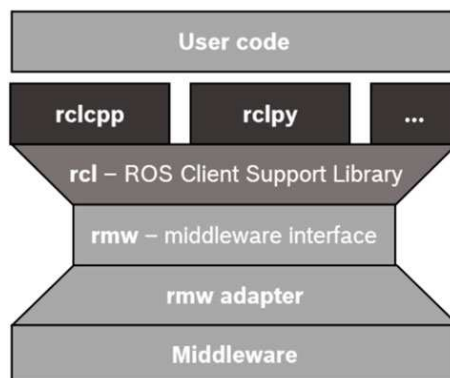


Figure 4.1: ROS 2 Client Library API Stack

¹<https://www.ros.org/>

- **Development tools:** ROS provides a suite of tools for managing and enhancing software development, including utilities for launching, introspection, debugging, visualization, plotting, logging, and playback.
- **Multi-machine support** ROS includes tools and libraries for retrieving, building, writing and executing code across multiple computers and OS.
- **Community and governance:** ROS is supported by an active and extensive global community, coordinated primarily by Open Robotics², which serves as the central organization for its development and maintenance.

²<https://www.openrobotics.org/>

4.1 Structure of ROS

The official ROS documentation³ provides comprehensive information about the system. All communication in ROS is handled through the **ROS Graph**, a peer-to-peer network of processes (called nodes) that execute concurrently and exchange information with one another. Some components of this graph are described below. Note that some of the functionalities described are specific to ROS 2 and are not available in ROS 1.

- **Nodes:** A node is a modular process that should performs a specific task. Nodes communicate with each other through topics, services, actions and parameters. In ROS 2, nodes can be implemented as **LifecycleNode**, which provide a managed state machine with defined transitions for startup and shutdown, enabling more deterministic behavior.

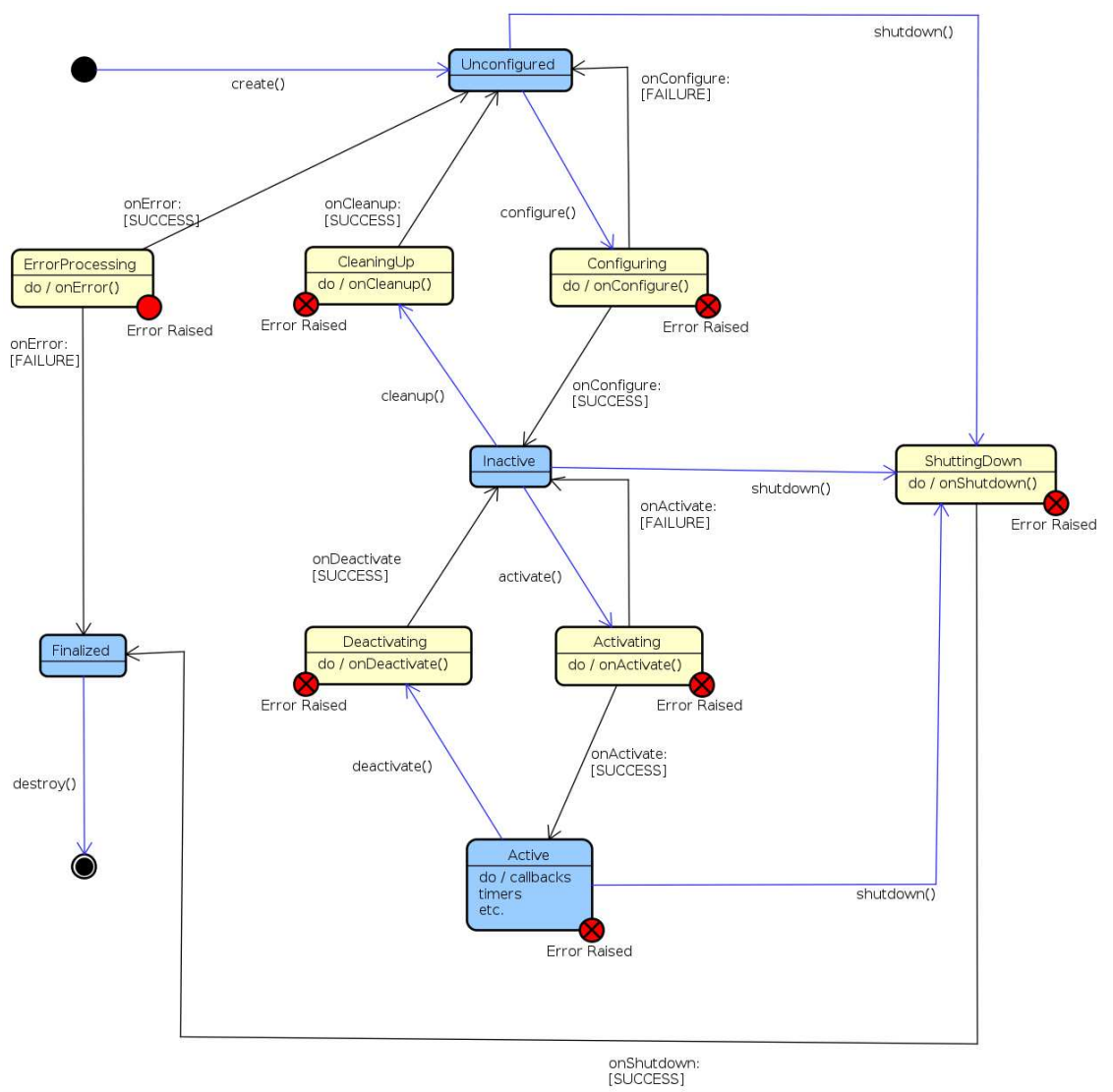


Figure 4.2: State Machine of LifecycleNode

Nodes can also be instantiate into process using the **Composition** feature,

³<https://docs.ros.org/en/rolling/index.html>

which also allow bypassing middleware overhead through intra-process communication, reducing RAM and CPU usage while improving goodput and latency[10].

- **Topics:** Topics are used for communication between nodes to share streams of messages in a publisher-subscriber structure.

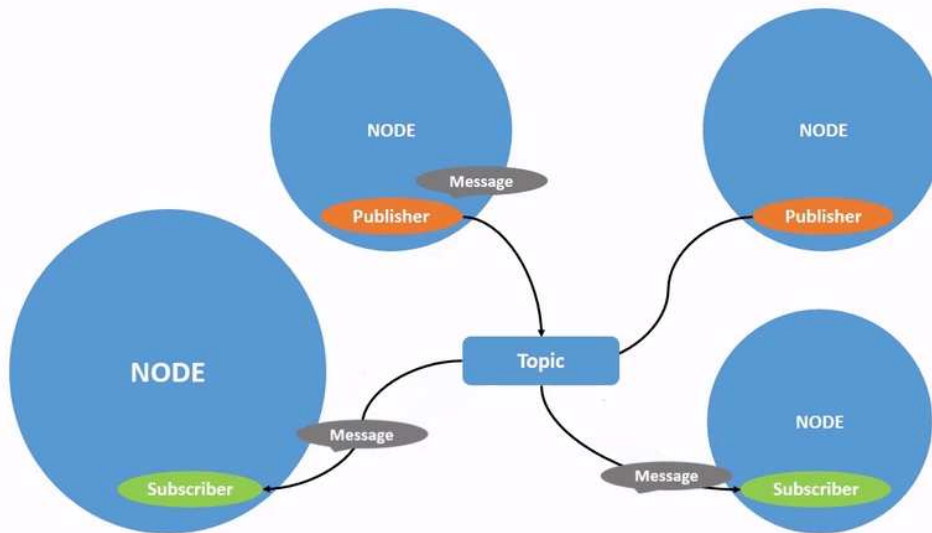


Figure 4.3: Topic process

- **Service:** Services are based on the request-response model. When a node with a service client sends a Request Message to a node with a service server, it receives a Response Message from that specific server.

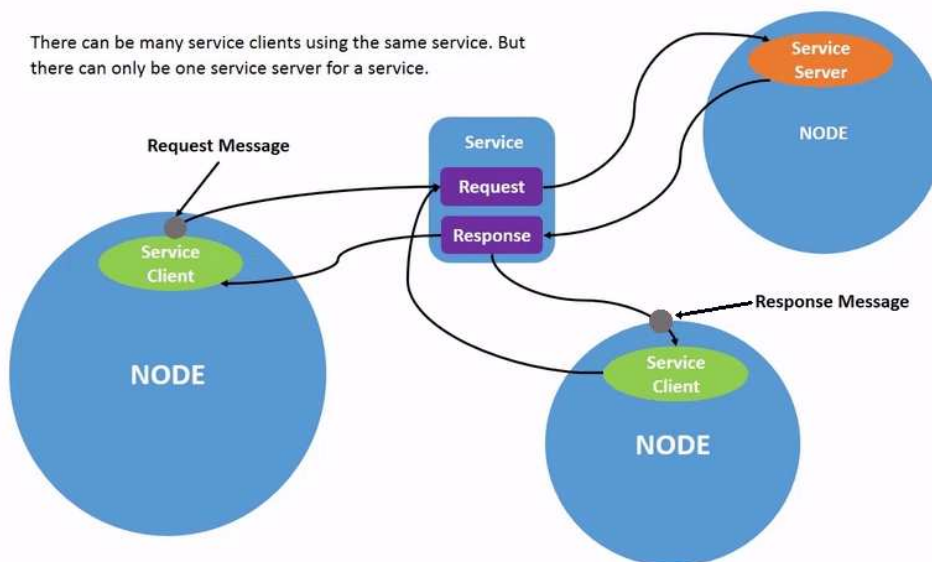


Figure 4.4: Service process

- **Parameters:** Parameters are configurations of a node that can be modified without changing the source code.

- **Actions:** Actions are designed to support long-running tasks that require feedback and the capability to interrupt when needed. They combine two services (goal and result) and a publisher-subscriber mechanism (feedback). A node sends a goal request and receives an acknowledgment, then starts publishing feedback messages while executing the task, and finally the action server responds with the result.

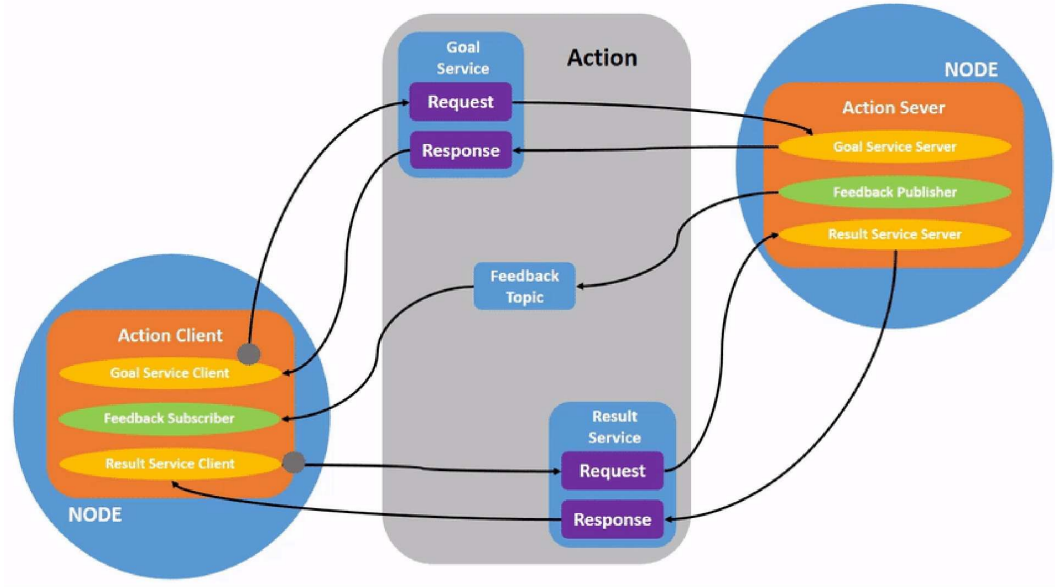


Figure 4.5: Action process

4.2 Tools of ROS

This section describes the tools available in ROS for developing robotic systems:

- **Launch:** A file structure that allows the simultaneous start of multiple nodes with a set of parameters.
- **rqt:** A graphical user interface (GUI) tool for ROS. It includes a series of plugins that facilitate development, monitoring, and diagnostics of the ROS system.
- **Rviz:** A 3D visualization tool for the Robot Operating System. It allows visualization of sensor data and state information from ROS.

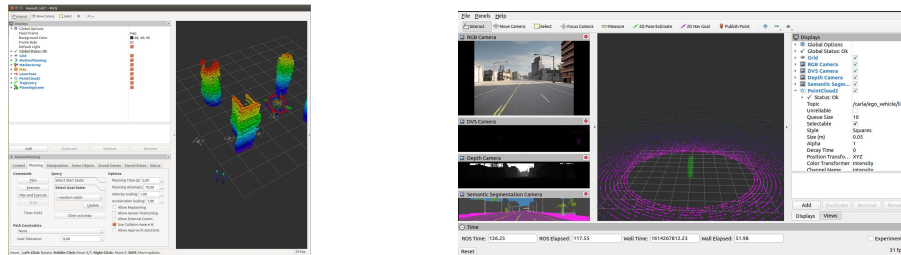


Figure 4.7: Examples of visualization of ROS data with Rviz

- **roscdep:** A command-line utility for installing system dependencies required by ROS packages.

- **sros2**: A security extension for ROS 2 that provides tools and instructions to apply security policies to DDS (Data Distribution Service)[18].

Another important category of tools that are integrated with ROS is simulation environments. These environments allow developers to test their code on simulated robots under various conditions, which helps identify bugs that could potentially harm a real robot or its environment. Simulation tools provide realistic physical behavior for all components of a robotic system. According to the ROS documentation, two main simulation tools are commonly used: Gazebo and Webots. Among them, Gazebo is also maintained by Open Robotics and is widely adopted in many projects, including the one discussed in this thesis.

Gazebo offer:

- **ROS Integration**: A bridge that converts Protobuf messages to ROS messages, enabling communication between Gazebo and ROS.
- **Performance tools**: Features for distributed simulation across servers, dynamic asset loading, and adjustable time steps to optimize simulation performance.
- **Realistic simulation**: Support for a variety of sensors, including their noise models, within a 3D world rendered using OGRE 2.1⁴ and powered by multiple physics engine, the default one is DART⁵.
- **Extensibility**: A modular design that supports plugins, allowing developers to run custom code that interacts directly with the simulation environment.

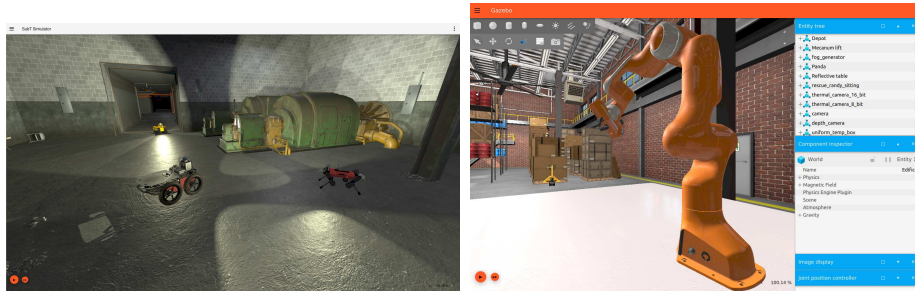


Figure 4.9: Examples of Gazebo

⁴<https://wiki.ogre3d.org/Home>

⁵<https://dartsim.github.io/>

Chapter 5

Introduction to NAV2 and Google Cartographer

Multiple tools are available to enable a robot to move autonomously within an environment. One of the most complete systems is Nav2¹ [12], the successor to the ROS 1 Navigation Stack.

5.1 Description of Nav2

Nav2 provides a comprehensive set of tools for perception, planning, control, localization, and visualization, allowing any robot to navigate autonomously and reliably, even in complex environments[15]. It enables the construction of an environmental model from sensor data, dynamically calculating the optimal path to avoid obstacles, setting motor velocities and executing navigation tasks using Behavior Trees.

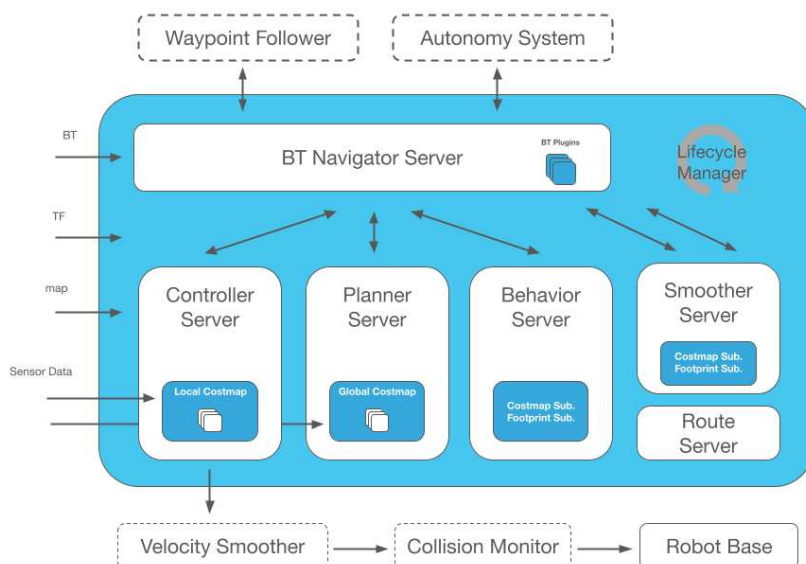


Figure 5.1: Nav2 architecture

¹<https://nav2.org/>

Nav2 introduces several improvements over ROS 1, including:

- **Behavior trees**^[4]: Nav2 uses Behavior Trees to structure and execute tasks in a human-readable and modular way. These trees enable the creation of complex systems and can be tested with various tools. The BehaviorTree.CPP² library is used to implement them, and it allows linking different trees for flexible behavior management.
- **nav2_util LifecycleNode**: This is a wrapper around the standard `rclcpp_lifecycle::LifecycleNode` described earlier 4.1. It adds a bond connection to the Lifecycle Manager node, which monitors the state of each node and can safely transition nodes through their lifecycle states in the event of a failure.
- **Action Server**: Described in 4.1, Nav2 uses action servers for Behavior Tree interactions, such as `NavigationToPose` (navigate to a specific location) and `NavigationThroughPoses` (navigate through multiple waypoints).

Nav2 is composed of several key nodes that work together to control the robot:

- **Controller Server**: Handles movement requests. It uses plugin-based modules such as a progress checker (to verify if the robot is making progress), goal checker (to determine if the robot has reached its destination) and a controller (to generate velocity commands using the **local costmap** while avoiding collisions).
- **Planner Server**: Computes the global path to the destination using sensor data and a plugin-based global planner loaded with the **global costmap**.

²<https://www.behaviortree.dev/>

- **BT Navigator Server:** Implements the NavigateToPose, NavigateThroughPoses, and other task interfaces. It is a Behavior Tree-based implementation of navigation that is intended to allow for flexibility in the navigation task and provide a way to easily specify complex robot behaviors.

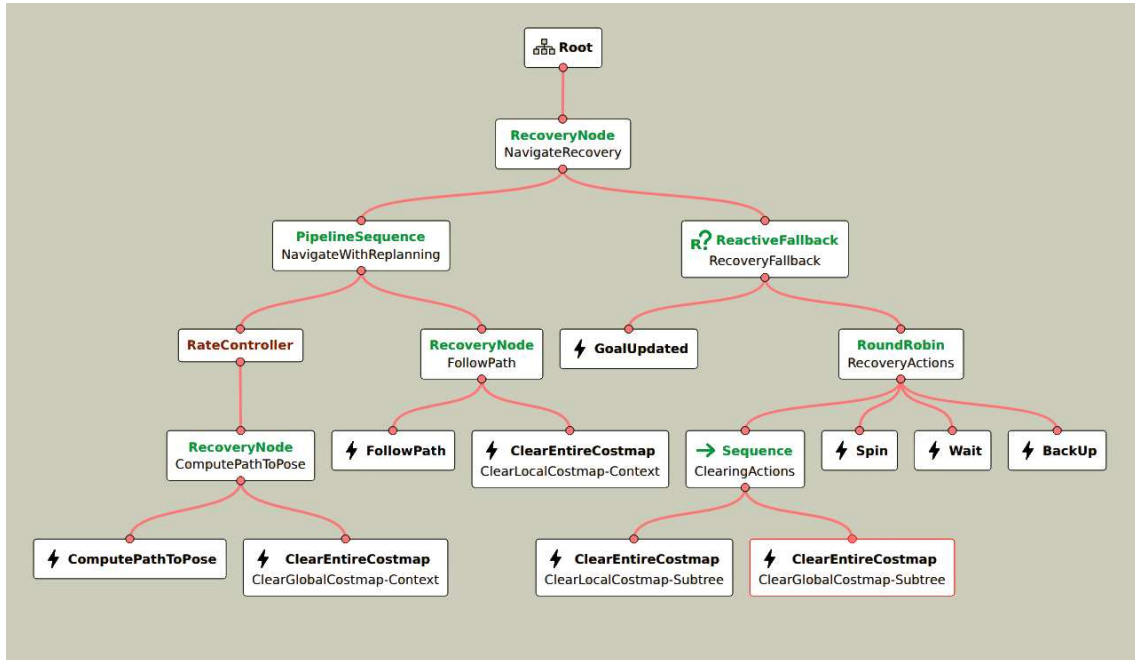


Figure 5.2: Humble Nav2 BT navigation `navigate_to_pose_w_replanning_and_recovery.xml` loaded with Groot2

- **Smoother Server:** Smooth the path minimizing sharp turns and rapid rotations.
- **Behavior Server:** Handles specific behavior actions like spinning, backing up, and waiting.
- **Waypoint Follower:** An example of orchestrated control (an **Autonomy System**), this node takes a list of waypoints and uses the NavigateToPose action server to move between them, optionally performing custom tasks (e.g., taking a photo) at each waypoint through plugins.
- **Route Server:** Calculates routes through a predefined navigation graph instead of using free-space planning.
- **Velocity Smoother:** Smooths velocity, acceleration, and deadband signals sent to the robot to ensure safe and smooth motion.
- **Costmap** is a representation of the environment using a regular 2D grid of cells with a cost (unknown, free, occupied, or inflated) and is used for generate the global plan or to compute local control efforts.

And require the following information for navigation:

- **state estimation** of the robot that generate a suitable TF tree, a tree structure of coordinate frames generated using the transform library[5], based on

the REP 105, that is should provide this structure: `map → odom → base_link`. According to the REP 105 they are defines as:

- `map` : A world-fixed frame that may change in discrete jumps. It's useful for long-term global reference but not suitable for short-term localization.
- `odom` : A continuous world-fixed frame that may drift over time. It provides short-term local reference.
- `base_link`: A robot fix frame.

The `odom → base_link` link can be estimated using various sensors. For improved accuracy, we use the `robot_localization` package[14], which provides nonlinear state estimation using Kalman Filter fusing the robot's odometry and the IMU data.

The `map → odom` link can be generate with a global positioning system like Simultaneous Localization And Mapping (SLAM) method[3] or using the one provided by Nav2, Adaptive Monte-Carlo Localization (AMCL).

- **map** can either be preloaded or built dynamically using a SLAM method. One SLAM method systems is Google Cartographer, which will be described in the next section. It is also the SLAM method that was used to generate the map we use for navigation.

5.1.1 Cartographer

Cartographer[8] is a SLAM method that operates using two main subsystems: a **Local SLAM** (called **Frontend**) that build a series of locally consistent **submaps** from successive scans and from the pose extrapolator with the possibility that it can drift during acquisition and a **Global SLAM** (called **Backend**) that reduces drift by performing loop closure—detecting when the robot has returned to a previously visited location. It uses scan matching against submaps and applies global optimization to align all submaps consistently and build a globally accurate map. Cartographer is capable of achieving mapping accuracy of 5 cm, but it requires extensive parameter tuning to reach optimal performance.

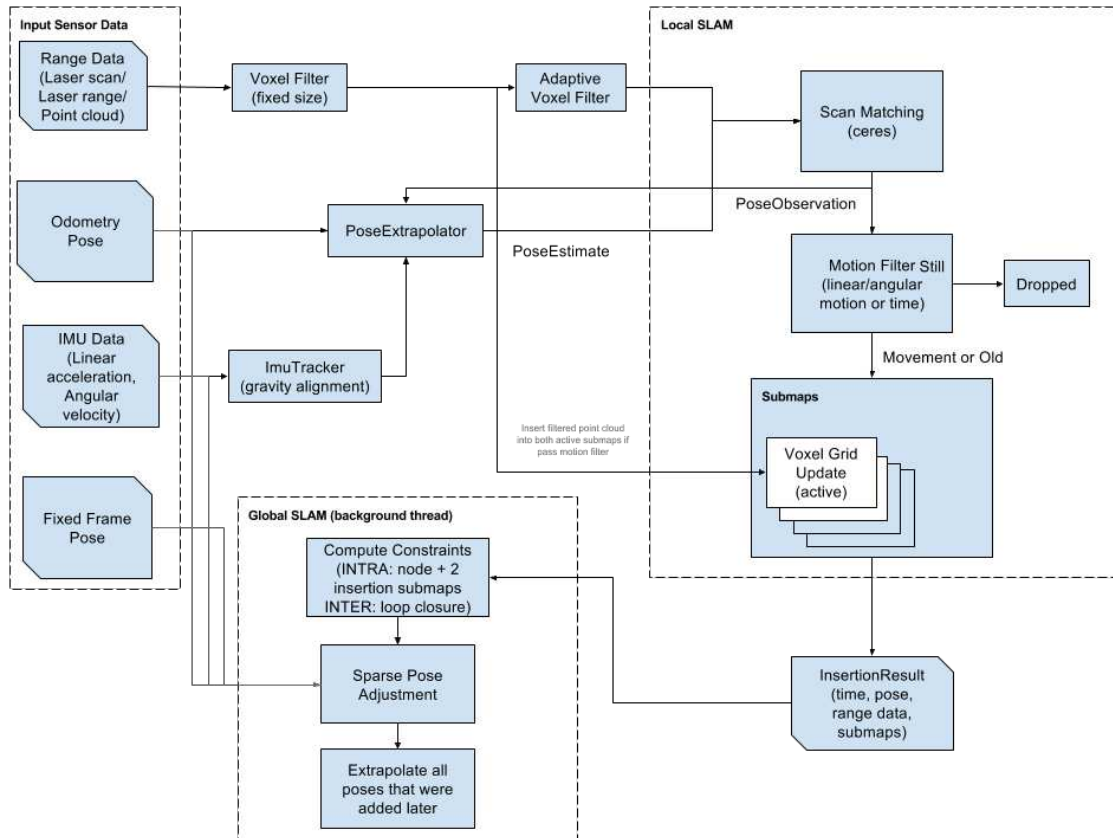


Figure 5.3: Cartographer Algorithm

Chapter 6

Review of related work on porting navigation stack from ROS 1 to ROS 2

The widespread adoption of the Robot Operating System (ROS) has introduced new use cases that were not originally considered during its development. Initially created as an academic project, ROS encountered limitations as users began applying it to more complex scenarios such as fleet management, deployment on embedded boards and microcontrollers, real-time systems, unreliable networks, and industrial applications. These emerging requirements highlighted architectural constraints in ROS 1, ultimately necessitating a complete redesign, which led to the development of ROS 2. Comprehensive documentation for ROS 2 is available at <https://design.ros2.org/>. Many companies have adopted the ROS 2 ecosystem, noting improvements in development workflows[11] and in specific subsystems like in Nav2[15]. New initiatives, such as micro-ROS¹, aim to bring ROS 2 capabilities to microcontrollers. Additionally, features like Node Composition offer improved performance optimization[10] while sros2 allow for a better security[18]. As outlined in these references, ROS 2 offers substantial advantages over ROS 1, making it the preferred framework for modern robotic applications. With this in mind, we will migrate the global planner using the official ROS 2 documentation² and various secondary sources³.

¹<https://micro.ros.org/>

²<https://docs.ros.org/en/rolling/How-To-Guides/Migrating-from-ROS1.html>

³https://industrial-training-master.readthedocs.io/en/melodic/_source/session7/ROS1-to-ROS2-porting.html

Chapter 7

Overview of open source fleet management systems, particularly Open-RMF

Currently, several fleet management systems are available:

- **Isaac Mission Dispatch**¹: A microservice-enabled fleet management system developed by NVIDIA that allows users to submit missions to multiple robots and monitor both robot and mission statuses. It provides connectivity between robots and the fleet management system but does not handle logistics such as task allocation or conflict resolution (e.g., intersecting robot paths). The implementation relies on the VDA5050 protocol, an industry standard for communication between cloud control services and mobile robots, and uses MQTT, a lightweight, publish-subscribe network protocol designed for devices with limited resources and bandwidth.
- **ROOSTER Fleet Management**²: An open-source ROS 1-based project that supports heterogeneous fleet management with task allocation, scheduling, and routing functionalities.
- **Open Robotics Middleware Framework (Open-RMF)**³: A free, open-source, and modular system that facilitates interoperability between multiple robot fleets and physical infrastructure such as doors, elevators, and building management systems.
- **Robofleet**⁴[16]: An open-source system based on ROS 1 that supports inter-robot communication, remote monitoring and remote tasking for heterogeneous fleets of ROS-enabled service robots. It is specifically designed to handle network variability and to incorporate security control features.
- **openTCS**⁵: (open Transportation Control System) is a free control system software for coordinating fleets of automated guided vehicles (AGVs) and mobile robots. It can control any automatic vehicle, independent of their specific

¹https://github.com/NVIDIA-ISAAC/isaac_mission_dispatch

²<https://github.com/ROOSTER-fleet-management>

³<https://www.open-rmf.org/>

⁴<https://github.com/ut-amrl/robofleet>

⁵<https://opentcs.org>

characteristics, with communication capabilities. Also, It can manage vehicles of different types and performing different tasks at the same time.

Other frameworks provide the necessary infrastructure to build a fleet control system:

- **Transitive Robotics**⁶: An open-source framework for full-stack robotics, designed to simplify the development of capabilities that connect robots, cloud services, and web front-ends through a dynamically updating shared data model. It addresses common challenges in building full-stack robotic applications, such as unreliable networks, intermittently offline robots, and cross-device versioning.
- **Robot Web Tools**⁷: A collection of tools that enable communication with remote robots via web technologies, facilitating remote monitoring and interaction.

Given that Open-RMF offers control over multiple fleets, enables structured interaction between them to achieve optimal outcomes, enable interaction with the building infrastructure, provides a comprehensive toolset for system development, is based on ROS 2 and is open source, it will be used as our fleet management.

7.1 Open-RMF description

Open-RMF is a flexible and modular toolkit built on ROS 2 that facilitates interaction between heterogeneous robot fleets. It employs standardized protocols to enable communication with infrastructure, environments and automation systems, optimizing the utilization of critical shared resources such as elevators, doors, narrow corridors, and other robots. By coordinating access to these resources, Open-RMF introduces system-level intelligence that helps prevent conflicts and ensures proper task and resource allocation. Figure 7.1 illustrates the architecture and operational workflow of Open-RMF.

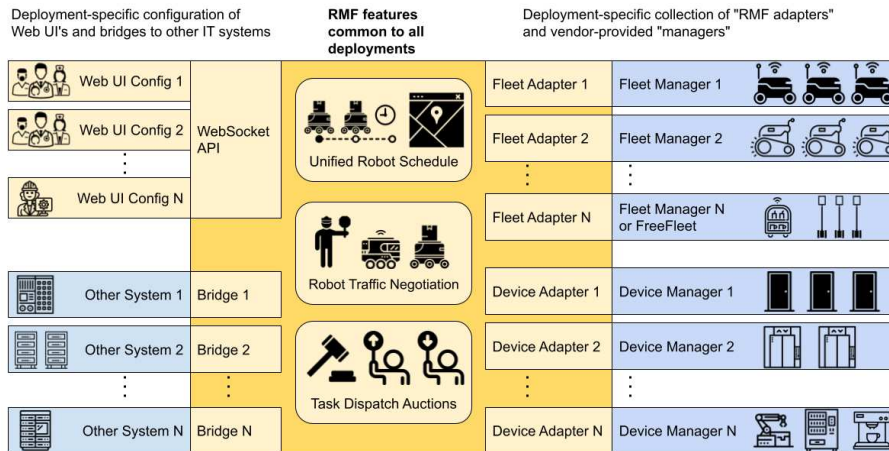


Figure 7.1: Open-RMF Structure

⁶<https://transitiverobotics.com/>

⁷<https://robotwebtools.github.io/>

The fleets that can be controlled through Open-RMF can be categorized into three types:

- *Full Control*: Fleets of this type provide full status updates and allow complete control over path planning and execution.
- *Traffic Light*: These fleets also provide full status updates, but only allow limited control, such as pausing and resuming operations. Open-RMF has control over certain behaviors, but not over the specific paths.
- *Read Only*: These fleets only provide status updates and cannot be controlled. Only one read-only fleet can exist in the system at any given time, as the presence of multiple read-only fleets would make it nearly impossible to avoid deadlocks or resource conflicts.

If a fleet does not belong to one of these categories, it falls under the classification of *No Interface* and is incompatible with Open-RMF, as this could lead to deadlocks in resource allocation.

The core of Open-RMF, known as `rmf_core`, is composed of several modular packages, each serving a distinct role within the system:

- `rmf_traffic`: Provides core scheduling and traffic management capabilities. It is based on a platform-agnostic **Traffic Schedule Database** and includes two main mechanisms for traffic deconfliction:

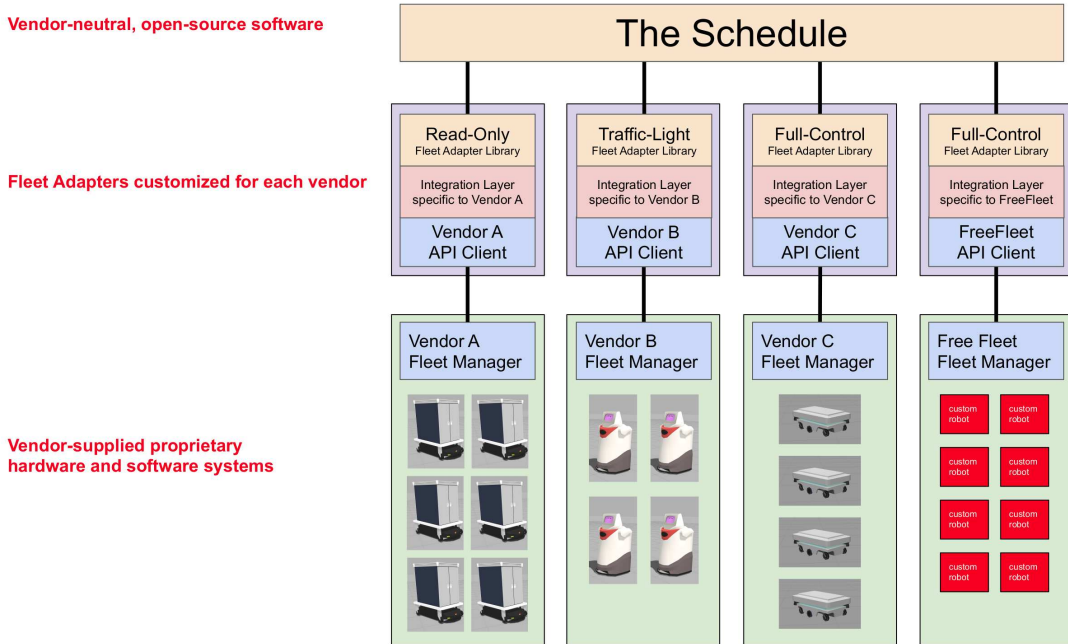


Figure 7.2: Open-RMF Schedule structure

- *Prevention*: When changes occur in the database (e.g., route cancellations), fleet managers can proactively replan routes to avoid conflicts.
- *Negotiation*: When a potential conflict is detected between two or more schedule participants, a conflict notice is issued to the relevant fleet managers. Each manager responds with a preferred path that accommodates other agents, and an external arbitration module selects the optimal path.

Conflict avoidance for Automated Guided Vehicles (AGVs) is implemented using a time-dependent extension to A* search. This algorithm considers both spatial and temporal dimensions, allowing it to generate conflict-free paths by accounting for the future movements of other agents. Support for Autonomous Mobile Robots (AMRs) is under active development.

- **rmf_task**: Contains APIs and base classes for defining and managing tasks in RMF. The framework natively supports three types of task requests:
 - *Clean*: For robots capable of cleaning floor spaces.
 - *Delivery*: For robots tasked with transporting items between locations.
 - *Loop*: For robots required to repeatedly navigate between two or more points.
- **rmf_dispatcher_node**: Contains the logic responsible for task allocation and management across multi-fleet systems. When a user submits a new task request, the dispatcher intelligently assigns it to the most suitable robot by querying each fleet adapter capable of performing the task. These adapters respond with bids indicating which robot is best suited, and RMF uses this bidding mechanism to determine the optimal assignment, as illustrated in Figure 7.3.

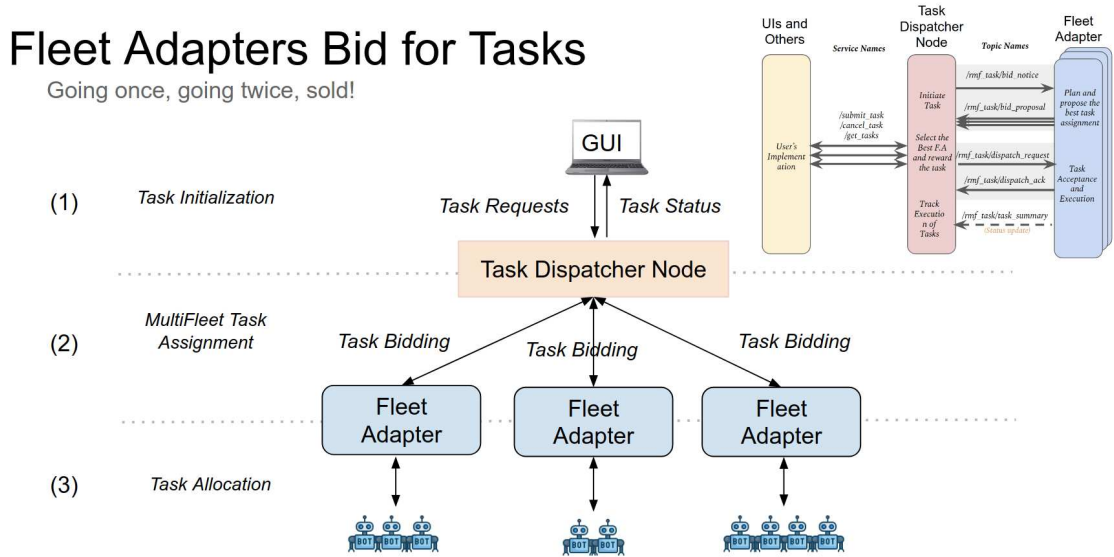


Figure 7.3: Open-RMF Bid Task

- **rmf_battery**: Provides battery consumption estimation models for different robot activities, aiding in energy-aware task planning.
- **rmf_obstacle**: Multiple packages that allow an interaction between the presence of obstacles or humans via sensors and the lanes of the fleet navigation graph.

- **rmf_ros2**: Supplies ROS 2 adapters, nodes, and Python bindings to integrate **rmf_core** components into ROS 2 distribute system, as show in the following picture.

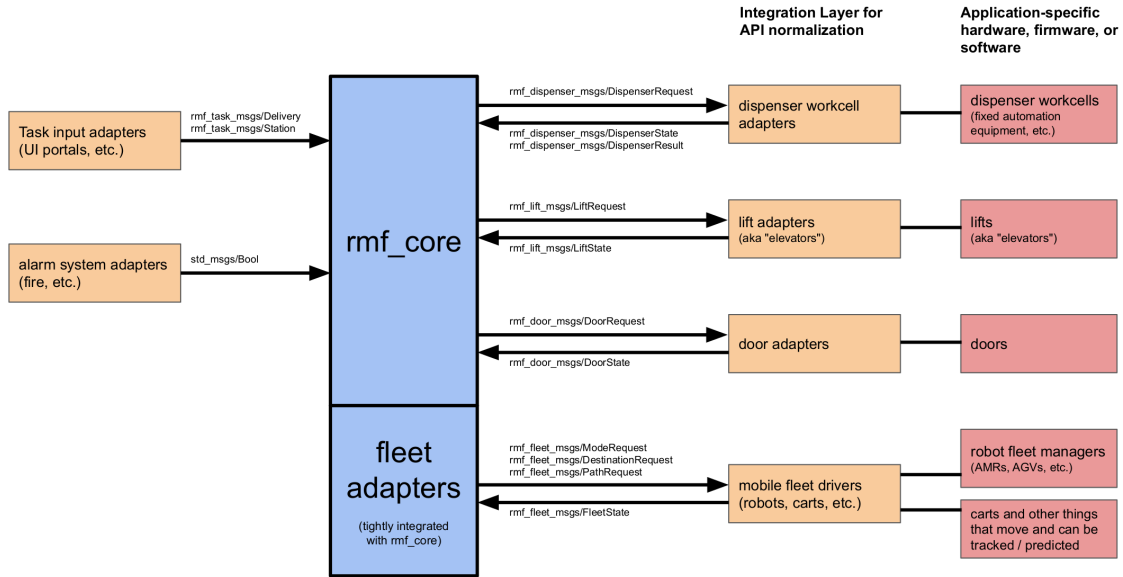


Figure 7.4: The communication needed to be handled by **rmf_ros2**

- **rmf_utils**: A collection of C++ utility functions and tools used by various RMF packages.

The following tools are available to support development with Open-RMF:

- **rmf_demos**: A collection of demonstration packages showcasing the capabilities of Open-RMF. These serve as a useful starting point for development and experimentation.

- **rmf_traffic_editor**: A graphical map editor used to define traffic patterns and interaction points (e.g., doors, lifts) that RMF uses for traffic management. It also supports the generation of Gazebo simulation worlds based on the designed maps.

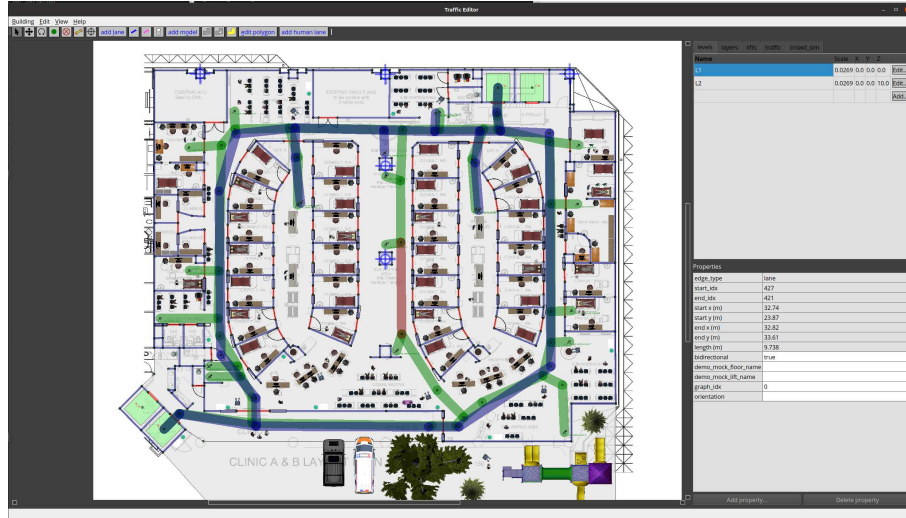


Figure 7.5: Traffic Editor

Another version of this software, **rmf_site**⁸, is currently undergoing active deployment.

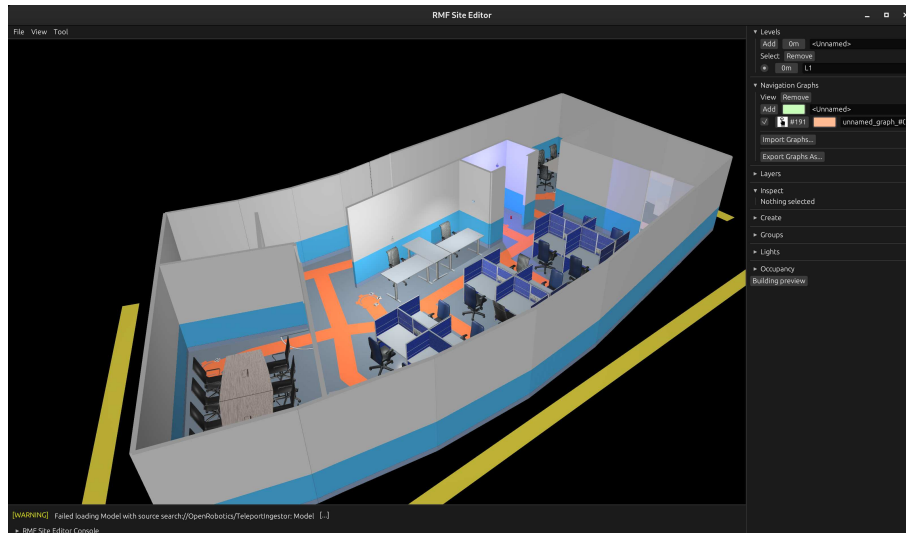


Figure 7.6: RMF Site Editor

⁸https://github.com/open-rmf/rmf_site

- **fleet_adapter_template**: An open-source fleet adapter template that enables the integration of standalone mobile robots into Open-RMF, allowing the creation of heterogeneous fleets.

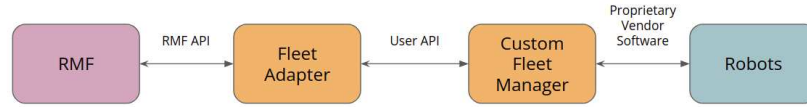


Figure 7.7: Fleet Adapter

From this package was developed **free_fleet** for controlling robots developed in ROS 1 and ROS 2.

- **rmf_visualization**: An RViz plugin that provides visualization and monitoring of the RMF schedule, enabling users to observe and interact with traffic and task execution in real time.

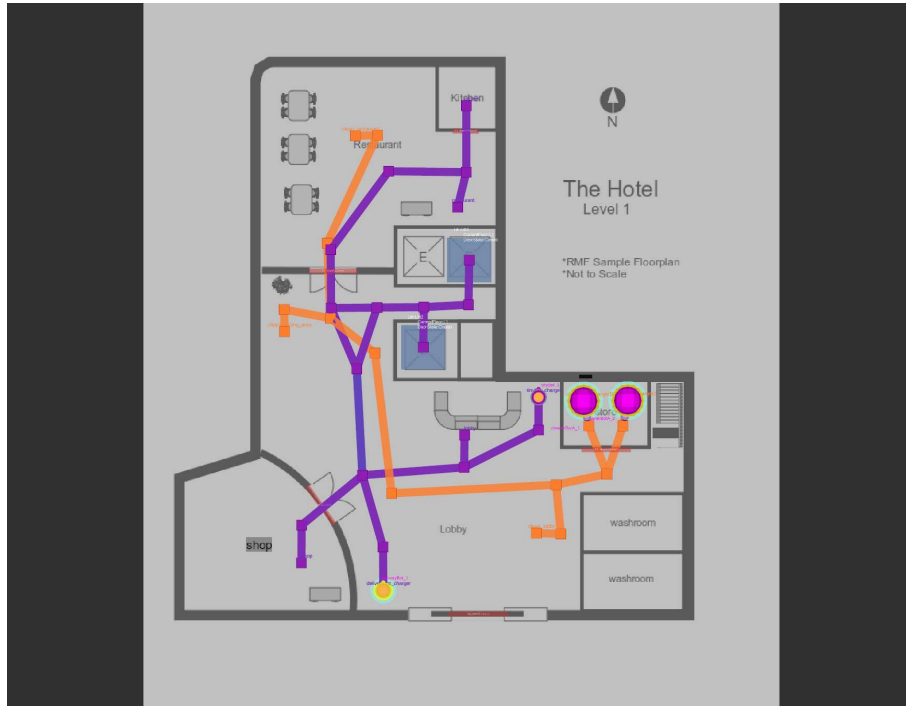


Figure 7.8: Visualization of RMF via RViz given by **rmf_visualization**

- **rmf-web** : A web-based application for visualization and control of the Robotic Middleware Framework (**rmf_core**). It provides a user-friendly interface for monitoring system state and interacting with fleet operations.

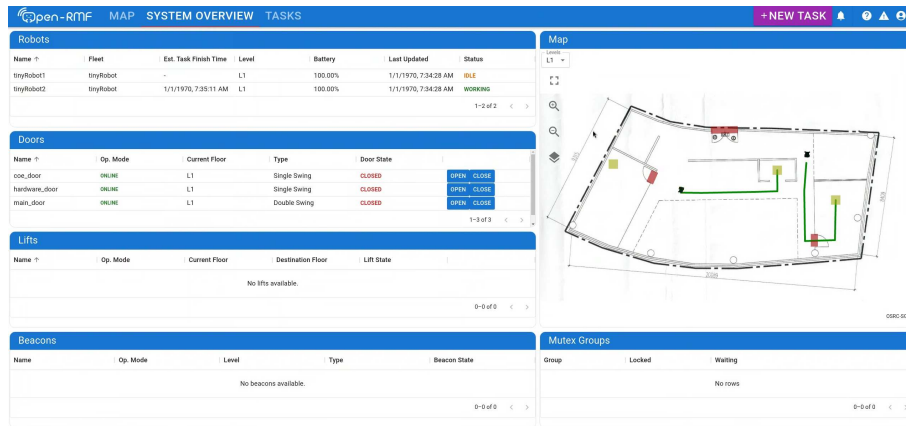


Figure 7.9: RMF Web

- **rmf_simulation**: A set of Gazebo plugins designed to simulate various aspects of building infrastructure—such as doors, lifts, and crowds—as well as robot behavior, including navigation and interactions with workcells.
- **door_adapter_template** and **lift_adapter_template**: Templates for writing a python based Open-RMF door and lift adapter.
- **Simulation assets**: Freely available 3D models and environment assets used to support simulation development and testing in Gazebo.

Part III

Methodology

In this part, we describe the entire procedure followed in the development of this project:

- The setup of our testing platform, based on the Rover Mini and a suite of onboard sensors.
- The configuration of the Nav2 navigation stack.
- The procedure used to port the global planner from ROS 1 to ROS 2, and its integration into the Nav2 framework.
- What we changed porting the old autonomous mobile robot Jobot from ROS 1 to ROS 2.
- The tools and methods used to integrate the system into Open-RMF.

All source code developed for this project is publicly available in the following GitHub repository: `michbelle/tesi_mag`⁹. The repository also includes the necessary components for simulating the system.

The simulation environment is containerized using Docker, which provides a consistent and OS-independent base system. The Docker image is built on top of `osrf/ros:jazzy-desktop-full` and includes all the packages developed for this thesis, as well as the required Open-RMF tools and its demo simulation.

⁹https://github.com/michbelle/tesi_mag

Chapter 8

Description of research platform and hardware used

The testing platform is a unmanned ground vehicle (UGV) called Mini that is a four-wheel drive (WD) skid-steering vehicle (see Figure 2.1). It has a maximum speed of 12.87 km/h and a payload of 45,36 kg. Its operational time ranges from 60 to 90 minutes under active usage, and up to 6 hours in idle mode. The manufacturer provides control and simulation code on GitHub¹, compatible with the Gazebo simulator. However, this code is officially tested only for the ROS 2 Humble distribution, which has therefore been selected as version of ROS used to control it. The UGV features a USB interface that allows connection to a PC or single-board computer for control purposes. In this work, a mini PC is used to control the UGV and also used to manage the entire navigation stack based on Nav2. The Mini PC used is a **Beelink SER3-E-16512EJ0W64PRO** with a Ryzen 7 3750H processor which is a quad-core 8-thread 2.3 GHz mobile processor boosting to 4.0 GHz with a Radeon RX Vega 10 Graphics. The mini PC draws power directly from the UGV, which supplies a 19V output, thereby reducing the overall operational time.



Figure 8.1: Mini PC used

To improve odometry estimation, an additional sensor is integrated: the **IMU Witmotion (WIT) JY901B**. This device is a 9-Axis Combined IMU/Magnetometer/Altimeter (measure linear accelerations, angular velocities, Euler angles, magnetic field, barometry, altitude) that communicates using the TTL/UART protocol. Sensor data are acquired using the Witmotion Sensors UART Connection

¹https://github.com/RoverRobotics/roverrobotics_ros2

Library[20], using a USB-to-serial converter and are published in the ROS 2 environment through a dedicated node, `witmotion_ros`, available on GitHub at `ElettraSciComp/witmotion_IMU_ros`².

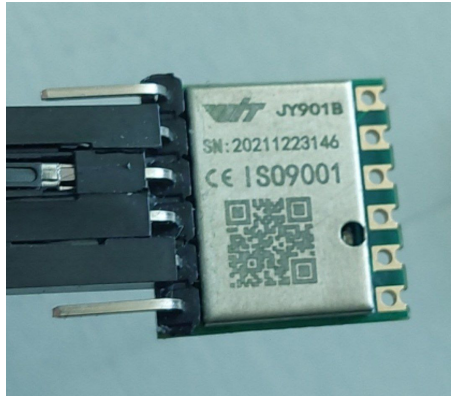


Figure 8.2: IMU used

A **2D laser scanner** is employed to provide environment perception and is used to compensate for odometry drift thanks the localization 5.1 which provide a transformation link between the `map` and `odom` TF frames. The 2D laser scanner provides a cost-effective solution for environmental perception, providing dense spatial information that enables obstacle detection and basic environment recognition. Compared to alternative technologies [2][13], it requires significantly less computational power, making it a practical choice for lightweight and resource-constrained robotic platforms. The Laser scan used is **RPLIDAR S1** a 360 degree 2D laser scanner (LIDAR) with a distance range of 40m on white object and 10m on black ones. It features a sample rate of 9.2kHz and a scan rate ranging from 8 to 15 Hz, resulting in an angular resolution from 0.313° to 0.587° , with an accuracy of $\pm 5\text{cm}$ and resolution of 3cm. The ROS 2 driver for the RPLIDAR S1 is available on Github at: `Slamtec/sllidar_ros2`³.



Figure 8.3: LIDAR used

²https://github.com/ElettraSciComp/witmotion_IMU_ros/tree/ros2

³https://github.com/Slamtec/sllidar_ros2

All components are physically mounted on the rover using custom 3D-printed parts, as shown in the following image.

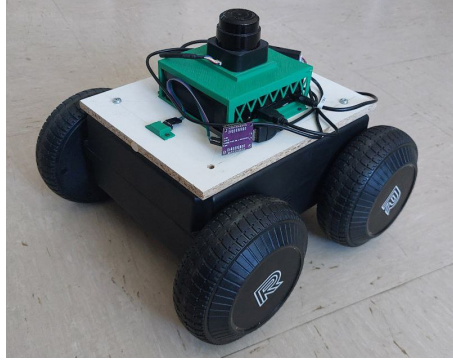


Figure 8.4: Robot used

To reflect the updated hardware configuration, the original URDF (Unified Robot Description Format) file provided by the manufacturer was modified. This ensures accurate sensor placement relative to the `base_link` frame, which is essential for proper operation of sensor-dependent ROS 2 nodes.

A 3D model of the robot, including all coordinate frames (TF), is shown in this picture:

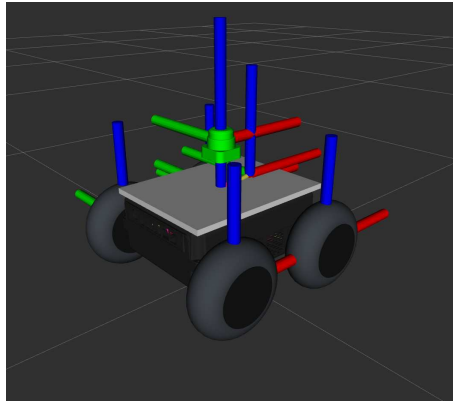


Figure 8.5: 3D model with TF

With the system fully assembled, the following ROS 2 launch files, from the package `mini_launchpad`, are used to retrieve data from the sensors connected to the robot's onboard mini PC:

- `0.1r_sensorLaunch.launch.py`: Launches the nodes responsible for publishing data from the IMU and LIDAR sensors.
- `0.2r_mini.launch.py`: Publishes odometry data based on the built-in encoders of the robot and subscribes to the `/cmd_vel` to move the robot.

While, for simulation, the following launch files, from the package `mini_simulation`, can be used for starting the simulation:

- `0s_ign_spawn_world_gazebo.launch.py`: That launch Gazebo with the world.
- `1s_ign_spawn_mini_gazebo.launch.py`: Spawn the model of the robot with all sensors in gazebo and launch the `ros_gz_bridge/parameter_bridge` node that is used to share simulated data between Gazebo and the ROS 2 environment.

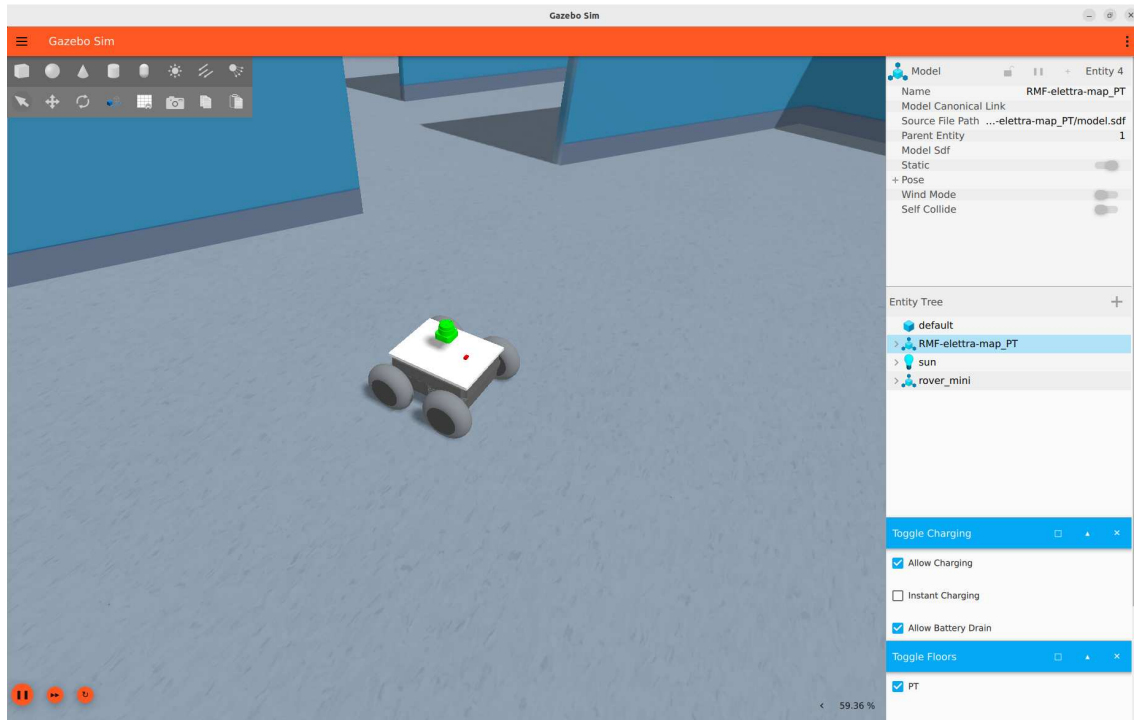


Figure 8.6: Model of the robot in Gazebo Ignition

Chapter 9

Steps taken to port the navigation stack from ROS 1 to ROS 2

In this chapter, we outline the requirements for the operation of Nav2 and provide a comprehensive overview of the packages utilized by Nav2. Particular emphasis is placed on the process of porting the Global Planner module from ROS 1 to ROS 2, with a detailed discussion of the required adaptations and the challenges encountered during the transition.

9.1 Odometry Estimation `odom` $\rightarrow$ `base_link`

As previously discussed, the raw odometry data provided by the rover are not sufficiently accurate, particularly in rotational movements. This is a well-known issue in odometry estimation for skid-steering vehicles, which tend to produce significant drift and errors when turning using only the data from the encoders.

To address this problem, the `robot_localization` package[14] was employed, specifically its `ekf_node` Node. This package enables the fusion of multiple sensor, even multiple instances of the same sensor type, and supports output in various ROS message formats, such as `nav_msgs/Odometry`. It also allows preprocessing of sensor messages prior to inserting into the estimation process. The filter performs continuous state estimation and is robust to intermittent sensor data loss, maintaining a coherent position estimate over time.

To launch the EKF node, the following launch file from the package `mini_launchpad` was wrote:

```
1<r/s>_robot_localizationEKF.launch.py
```

9.1.1 Configuration Odometry Estimation in `ekf_node`

As previously described and detailed in the node's documentation¹, we added the information about the sensors used to the configuration file. The principal information to include is the type of sensor and on which topic subscribe, `odom0:odometry/wheels` and `imu0:imu/data`, and, for each sensor, we need to choose which components of the state measurement data we want to fuse using the following array by setting the corresponding values to `true` for those we wish to include and `false` for those we

¹https://docs.ros.org/en/noetic/api/robot_localization/html/index.html

do not:

```
sensorX_config:[X,Y,Z,roll,pitch,yaw, $\dot{X}$ , $\dot{Y}$ , $\dot{Z}$ , $\dot{roll}$ , $\dot{pitch}$ , $\dot{yaw}$ , $\ddot{X}$ , $\ddot{Y}$ , $\ddot{Z}$ ]
```

Additionally, the process noise covariance matrix Q can be tuned. This matrix represents the noise we add to the total error after each prediction step. Generally, the larger the value of Q relative to the variance of a given variable in an input message, the faster the filter will converge to the value indicated by the measurement.

More information on the testing and the parameters analysis are described in the chapter Testing and Evaluation 12.2 .

9.2 Localization map $\rightarrow$ odom

In this section, we describe the process of acquiring a map and establishing the transformation between the `map` frame and `odom` frame using a localization method. This is necessary because, for long-term operation, the position of the robot in the world, using only odometry data, tends to accumulate errors and is not sufficiently accurate. Therefore, a localization technique is required to compensate for these errors.

9.2.1 Mapping

Although a map is already available, we will describe the process of constructing a map using the Simultaneous Localization And Mapping (**SLAM**) technique[3]. Below, we provide instructions for running the following SLAM packages: `slam_toolbox`[9] (GitHub: SteveMacenski/slam_toolbox²) and `cartographer_ros`[8] (GitHub: ros2/cartographer_ros³).

These packages can be launched from the `mini_launchpad` using the following launch files:

```
X<r/s>_mappingW<slam/cartographer>.launch.py
```

To save the generated map, use the following command, which works for both `slam_toolbox` and `cartographer_ros`. This will save the map as two files: a `.pgm` image and a `.yaml` metadata file:

```
ros2 run nav2_map_server map_saver_cli -f map
```

When using Cartographer, it is also possible to save the map as a `.pdstream` format with the following command:

```
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState  
  "{filename:  \"map.pdstream\", include_unfinished_submaps :false}"
```

²https://github.com/SteveMacenski/slam_toolbox/tree/ros2

³https://github.com/cartographer-project/cartographer_ros

9.2.2 Only localization

Since we have already a map we only need to focus on localization. The map in use was created using Cartographer by the robot Jobot while navigating through Building T of Elettra, which serves as an office building. The map is:

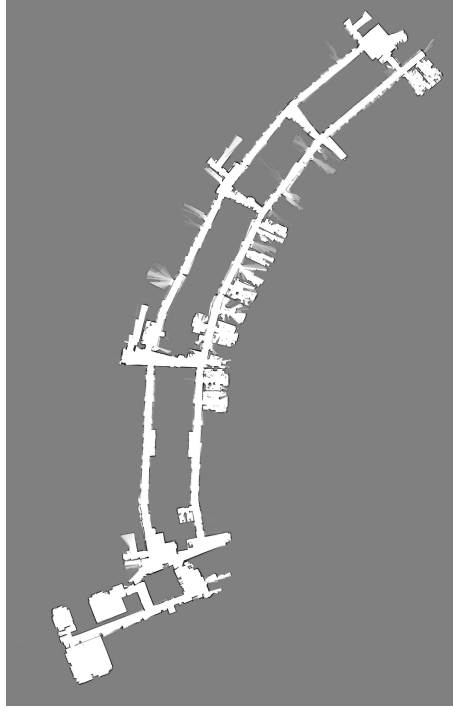


Figure 9.1: Map acquired with Cartographer

We initially attempted to use Cartographer solely for localization; however, due to the project's limited maintenance, we encountered difficulties in using it effectively. As a solution, we opted for the method provided by the Nav2 package, specifically `nav2_amcl` and the `amcl` node that contains a pure and fully-parameterized implementation of the grid particle filter localizer[7][17].

One limitation of this approach is the need to convert the map acquired with Cartographer from the `.pbstream` format into a `.pgm` image and `.yaml` metadata pair, which are required by the `amcl` node. This conversion can be performed using the following command:

```
ros2 run cartographer_ros cartographer_pbstream_to_ros_map \
  -pbstream_filename map.pbstream \
  -map_filename map.pgm \
  -resolution 0.05
```

Since certain walls in the Cartographer generated map were not solid black, due to the uncertainty of their position during mapping, the conversion process introduced errors that led to these areas to be misinterpreted as free space. As a result, the planner generated paths that passed through them. To address this issue, the walls were manually corrected using GIMP, an image editing tool, by modifying the `.pgm` map file.

Additionally, our planner is designed with the assumption that unknown spaces are not usable for planning. Therefore, we set the parameter `track_unknown_space` in the global costmap node to `true` (`false` by default) to achieve optimal results from our planner. The corrected map is shown below:

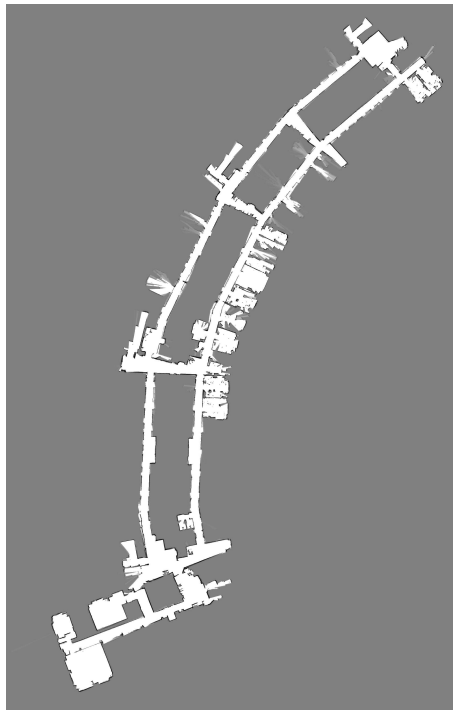


Figure 9.2: Map edited with Gimp and used

With the map properly adjusted, all necessary tools for localization are now available. We created the following launch file in the package `mini_launchpad` to initiate localization:

```
3<r/s>_L<amcl/Cartographer>.launch.py
```

At this stage, all requirements are met to utilize the Nav2 navigation stack.

9.3 Structure Navigation Stack Nav2

In this section, we will discuss the components used in the Nav2 navigation stack and describe the process of porting the Global Planner from ROS 1 to ROS 2.

9.3.1 Navigation Launch file

The launch file contains all the necessary processes required for the Navigation stack Nav2 to operate. It also implement the composition structure (4.1) and the lifecycle manager (4.1). Below is a list of the core components used, along with a brief

description of the plugins associated with each. Additional information can be found in this documentation⁴, and a migration guide is available in this documentation⁵:

- `nav2_controller::ControllerServer` that use the following plugins:
 - As `controller_plugins` use `nav2_mppi_controller::MPPIController`: a Model Predictive Path Integral (MPPI)[22] algorithm to track a path with adaptive collision avoidance or use the `dwb_core::DWBLocalPlanner` that use the Dynamic Window Approach [6]
 - As `progress_checker_plugins` use `nav2_controller::SimpleProgressChecker` that checks whether the robot has made positional progress.
 - As `goal_checker_plugins` use `nav2_controller::SimpleGoalChecker` that checks whether the robot has reached the goal pose.
- `nav2_smoother::SmootherServer` that use the following plugins:
 - As `smoother_plugins` use `nav2_smoother::SimpleSmoother` that takes an input path and applies a lightweight smoothing algorithm. It weights the initial path points and the smoothed path points to create a balanced result where the path retains its high level characteristics but reduces oscillations or jagged features.
- `nav2_planner::PlannerServer` that use the following plugins:
 - As `planner_plugins` use `dstar_global_planner::DStarGlobalPlanner` that is our global planner plugin and is a implementation of the D* informed incremental search algorithm with external optimizers[19].
- `nav2_behaviors::BehaviorServer` that use the following plugins:
 - As `behavior_plugins` use: `nav2_behaviors/Spin` performs an in-place rotation by a specified angle, `nav2_behaviors/BackUp` moves the robot backward by a set distance, `nav2_behaviors/Wait` is used to pause navigation for a specified duration, allowing time for temporary occlusions to clear before resuming path planning.
- `nav2_bt_navigator::BtNavigator` that use the following plugins:
 - As `navigators` use `nav2_bt_navigator::NavigateToPoseNavigator` and `nav2_bt_navigator::NavigateThroughPosesNavigator` that utilize the default behavior trees (BT) for navigation.
- `nav2_lifecycle_manager::LifecycleManager` that handle the lifecycle transition states for the stack in a deterministic way.

The simulation configuration file contains minor differences due to the use of different versions of ROS 2: Jazzy in the simulated environment and Humble on the physical robot. In the next section, we will describe the key steps taken to port the global planner `DStarGlobalPlanner` from ROS 1 to ROS 2.

⁴https://docs.nav2.org/setup_guides/algorithm/select_algorithm.html

⁵<https://docs.nav2.org/migration/index.html>

9.3.2 Porting ROS 1 DStarGlobalPlanner to ROS 2

As previously mentioned, the `DStarGlobalPlanner`⁶ is a implementation of the D* informed incremental search algorithm[1] with external optimizers[19] for the ROS 1 `move_base` package⁷.

Since the code is written using ROS 1, which is not compatible with ROS 2, it will need to be ported to the newer version. This process was divided into two main phases. In the first phase, we migrated the base structure of the package from ROS 1 to ROS 2, following these ROS 2 porting guidelines^{8 9}. In the second phase, we adapted the code to comply with the structure required by Nav2 planner plugins, based on the Nav2 documentation titled: "Writing a New Planner Plugin"¹⁰. The following subsections describe these two phases in detail. The first focuses on the structural migration to ROS 2 and the integration into the Nav2 framework. The second details the modifications needed to adapt the planner logic and interfaces to match ROS 2 conventions and the plugin architecture.

Porting code from ROS 1 to ROS 2

In the `package.xml` file, which describe the package as an organizational unit for your ROS 2 code, the following changes were made:

- `buildtool_depend : catkin` → `ament_cmake`: `ament_cmake` is the new build system for CMake based packages. Compared to `catkin`, it introduces many utilities for managing packages. More information is available in the documentation¹¹.
- `roscpp` → `rclcpp`: `rclcpp` is the new ROS client library for C++, built on `rcl`, which provides the foundation for language-specific client libraries. `rcl` communicates with `rmw`(ROS middleware interface), which handles the communication with DDS (Data Distribution Service).
- Updated package naming to follow the ROS 2 conventions.
- To inform `colcon` that the package uses `ament_cmake`, the build type must be explicitly specified in the `build_type`.

Subsequently, the `CMakeLists.txt` file was updated with the following major changes:

- Package names were updated to reflect ROS 2 naming conventions.
- C++ standard was set to 17.
- `find_package()` is required to be called for each package.
- `ament_export_dependencies()` is used to declare runtime dependencies.

⁶https://github.com/ElettraSciComp/DStar-Trajectory-Planner/tree/ros2_port

⁷https://wiki.ros.org/move_base

⁸<https://docs.ros.org/en/rolling/How-To-Guides/Migrating-from-ROS1.html>

⁹https://industrial-training-master.readthedocs.io/en/melodic/_source/session7/ROS1-to-ROS2-porting.html

¹⁰https://docs.nav2.org/plugin_tutorials/docs/writing_new_nav2planner_plugin.html

¹¹<https://docs.ros.org/en/rolling/Concepts/Advanced/About-Build-System.html>

- The package must be declared in ROS with `ament_package()`.

Other minor changes are:

- Log calls were updated from `ROS_` to `RCLCPP_`
- Message type structures changed (e.g., `geometry_msgs::PoseStamped` → `geometry_msgs::msg::PoseStamped`).
- The structure of the C++ program now required the use of smart pointer.

Additional changes specific to the ROS 2 Navigation stack Nav2 are discussed in the following section.

Porting code from ROS 1 Navigation stack to Nav2

The following changes were made to port the planner to the Nav2 framework:

- Since the planner is a **global planner**, it now inherits from the new abstract interface `nav2_core::GlobalPlanner`. As a result, we updated the `global_planner_plugin.xml` file to change the `base_class_type` accordingly.
- As a plugin based on `nav2_core::GlobalPlanner`, the planner must implement five pure virtual methods required for proper operation: `configure()`, `activate()`, `deactivate()` and `cleanup()`, `createPlan()`. Below is a description of each method:
 - `configure()`: it's called when the node enters `on_configure` state. This method is responsible for declaring ROS parameters and initializing the planner's member variables. It takes the following input parameters:
 - * `rclcpp_lifecycle::LifecycleNode::WeakPtr` A weak pointer to the base lifecycle node.
 - * The planner's name as a `std::string`.
 - * A share pointer to the `tf` buffer
`std::shared_ptr<tf2_ros::Buffer>`.
 - * A shared pointer to the costmap
`std::shared_ptr<nav2_costmap_2d::Costmap2DRos>`.
 - * For jazzy also: `std::function<bool()>` `cancel_checker` that allow for the cancellation of the current planning task.

This replace the `initialize` method and the class initialization used in ROS 1.

- `activate()`: Called when the node enters `on_activate` state, used for implement operations which are necessary before planner goes to an active state. In our case, we allocate memory and fill our internal costmap `state_map`, which is used by the D* (`dstar`) algorithm.
- `deactivate()`: Called when the node enters `on_deactivate` state. This method is used for implementing operations which are necessary before planner goes to an inactive state. In our case, this method is not used and is left empty.

- `cleanup()`: Called when the node enters `on_cleanup` state. This method is used for cleaning up resources which are created for the planner. In our case we destroy the allocated memory used by `state_map` and `dstar`.
- `createPlan()`: Invoked when the planner server requests a global plan between a start and goal pose. It required the start and goal position as `geometry_msgs::msg::PoseStamped` and return a path `nav_msgs::msg::Path`. It replace the `makePlan` used in ROS 1.
- At the end of the C++ code, we added the following lines to export the planner as a plugin:

```
#include "pluginlib/class_list_macros.hpp"
PLUGINLIB_EXPORT_CLASS(<type of the plugin class>, <type of the base class>);
```

- Because it is a plugin, it is necessary to provide a description using a plugin declaration file. This is done by creating a file named `global_planner_plugin.xml` in the same directory as the `package.xml`. The `global_planner_plugin.xml` file specifies the plugin's class type and the base class it inherits from, following the format required by the pluginlib system. Then we edit the CMake file as described in the documentation¹².

Building the project

The system was built using the new build tool, `colcon`¹³. It is a command line tool to improve the workflow of building, testing and using multiple software packages. It automates the process, handles the ordering and sets up the environment to use the packages.

After successful compilation, the global planner is ready for use within the Nav2 framework.

¹²<https://docs.ros.org/en/rolling/Tutorials/Beginner-Client-Libraries/Pluginlib.html>

¹³<https://github.com/colcon>

Chapter 10

Jobot porting procedure to ROS 2

Elettra has also a Jobot, a 2WD rover still based on ROS 1, it was decided to port it to ROS 2 in order to effectively evaluate the use of Open-RMF.

10.1 Jobot Description

Jobot is a 6-wheeled AMR with a 2WD configuration. It is equipped with a mini PC that collects data from an RPLIDAR A2 laser sensor and acceleration data from a serial bus connected to an Arduino microprocessor, which is linked to an IMU. An image of the robot is available in 2.2. The steps taken to port the code in ROS 2 are described in the following section. When porting the system we used the same design for navigation and control that was used when build the first time[21].

10.2 What changes where made in the code

The code use the serial ROS 1 library to communicate to the Arduino and retrieve the data from the encoders and the IMU. This library is no longer available in ROS 2, so to avoid rewriting the structure of the Jobot code, we utilized the same library ported to ROS 2, which is available on GitHub: [iwelch82/serial-ros2](https://github.com/iwelch82/serial-ros2) . The retrieved data is used to publish the odometry pose estimation of the robot, while a secondary node handles the IMU data.

The node that publish the IMU data was updated to ROS 2 by simply using the base node logic, whereas the main node was rewritten using the lifecycle structure. This logic is employed to configure the node during the `on_configure` method, and if it fails or a request for deactivation or cleanup is made, it stops and cleans everything. This structure allow to remotely control the passage through the states, for now and in our case it will be connected to an automatic Node manager like the one used in Nav2, the `lifecycle_manager` in `nav2_lifecycle_manager`. This node check all the states of the nodes that are connected to it and sequentially configures and activates them. If a crash occurs or if we stop a specific node's execution, the lifecycle communicates the change of state and activates a procedure to stop the connected nodes. This node is only called in the launch file.

In addition we updates the custom messages to ROS 2 that the Jobot use internally.

Once everything is configured, the node starts to publish the robot's state odometry

but does not publish static transforms between the sensors. These transforms are manually published using the `static_transform_publisher` provided by `tf2_ros`.

To minimize changes, the software is built inside a Docker container, and all applications are launched from it.

This, along with the same software used for navigation, enables this platform to function as an autonomous mobile robot (AMR).

10.3 Create a model for the simulation

For simulation purpose, we need to create a model of the Jobot in the URDF format, which will be used by Gazebo to build a 3D representation and generate data from the sensors equipped on it.

Since Jobot has a 2WD configuration, we will add two cylinders as wheels, which will be controlled by the Gazebo plugin `ignition::gazebo::systems::DiffDrive`. This plugin allows the robot to receive velocity commands and move accordingly. Next, we add the sensors `gazebo_typeIMU` and `gpu_lidar` to simulate the IMU and the lidar on the robot. This procedure is similar to the one used for the Mini Rover from the Rover Robotics. At the end we obtain the following model:

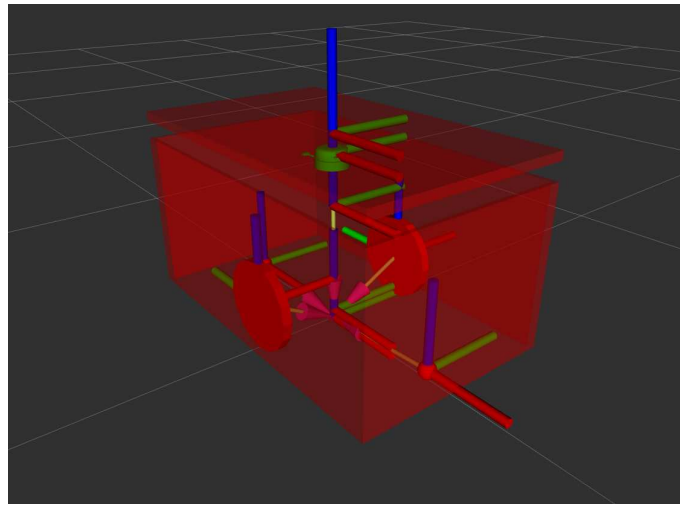


Figure 10.1: Jobot robot URDF model

To launch the sensors and the robot, a naming structure similar to that used for the Rover Mini is implemented for the Jobot. The following packages have been created for the Jobot:

- `jobot_driver_ros2`: This package contains the code for controlling the Jobot.
- `jobot_launchpad`: This package is responsible for launching Nav2.
- `jobot_msgs`: This package includes the custom messages used by Jobot.
- `jobot_simulation`: This package contains all the code necessary to simulate the Jobot in Gazebo.

Chapter 11

Integration of the navigation stack with Open-RMF

To integrate a fleet in Open-RMF, we first need information about the building's infrastructure, such as automatic doors or lifts that can be controlled and the **route maps** that robots can use. **Route maps** are essential for predicting the paths robots will take, enabling Open-RMF to perform effective **traffic management** and avoid conflicts. All these information have to be stored in a file that can be generated using the **traffic editor tool**^{1 2}, a tool provided by Open-RMF. The following section describes how the Traffic Editor works and how we used it in our implementation.

11.1 Traffic Management - Route Maps Generation

To begin describing the environment for Open-RMF, we created a new project in the Traffic Editor by generating a `.building.yaml` file. We then imported both the map produced by Cartographer and the floor plan of the facility. After importing the map, we defined the positions of key elements and planned the traffic flow for the floor, following best practices as outlined in the Open-RMF documentation³. The following components of the Traffic Editor were used to encode the necessary environment data:

- **vertex**: They are waypoints or a part of the description of walls, doors or else. Each vertex can have several properties:
 - `is_holding_point` : Indicates that the robot is allowed to wait at this waypoint for an indefinite period of time.
 - `is_parking_spot`: Designates a parking location for a robot.
 - `is_passthrough_point`: A waypoint where the robot should not stop.
 - `is_charger`: Identifies a charging station.
 - `is_cleaning_zone`: Used to indicate that this waypoint belong to a cleaning zone for the `Clean` task.

¹https://github.com/open-rmf/rmf_traffic_editor

²<https://osrf.github.io/ros2multirobotbook/intro.html>

³https://osrf.github.io/ros2multirobotbook/integration_nav-maps-strategies.html

- **dock_name**: Used for triggering docking behavior via RMF.
- **pickup_dispenser**: Specifies a dispenser workcell for **Delivery** tasks.
- **dropoff_ingestor**: Specifies an ingestor workcell for **Delivery** tasks.
- **spawn_robot_type**: Used in simulation to spawn a robot model.
- **spawn_robot_name**: Used in simulation to identify a spawned robot.
- **measurement**: Connects two vertex and is used to set the scale of the drawing. In our case, we used the width of a door.
- **door**: Connects two vertex and is used for interacting with the real world and simulate doors. If a robot encounters a door on its route, RMF sends a command to open it. When generating the simulation world model, a controllable door model is automatically added.
- **traffic lane**: Connects two waypoints and define the paths that fleets will follow. Lanes can be grouped, assigned to specific fleets, and configured as unidirectional or bidirectional. An optional "orientation" setting can be specified to require robots to move forward or backward along the lane. During planning, Open-RMF assumes that all traffic lanes are straight lines.
- **fiducials**: Used for multi-floor alignment and as anchors for the ROS 2 map. They provide spatial reference points between floors.
- **lift**: Is used for interacting with the real world and simulate elevators. If a robot goes to a lift, RMF sends a command to call it at the floor the robot is and, when the robot reach the waypoint inside it, then RMF send the lift to the designed floor. When generating the simulation world model, a controllable lift model is automatically added.
- **walls**: Connect two waypoints to define the location of a wall in the environment. When generating the simulation world, a wall is created between these points.
- **floor**: Add the ground plane for robots in the simulation world. This component can be modified to include holes (e.g., for elevator shafts).
- **environment assets**: Adds Gazebo models of various furniture and objects to the simulation, helping to create a realistic environment for testing and validation.

In the process of designing route maps, several strategic choices were made:

- Only one robot is allowed in a hallway at a time.
- Holding points (leaf nodes) were added along corridors or near their entrances and exits to allow one robot to yield to another already in the hallway. For certain task-specific waypoints, it is recommended to assign a very low speed limit to discourage RMF from using them as generic holding points.
- Mutex groups were defined on lanes and waypoints to ensure exclusive access, avoiding deadlocks by allowing only one robot to occupy a mutex-assigned group at any time.

At the end we obtained the following configuration map:

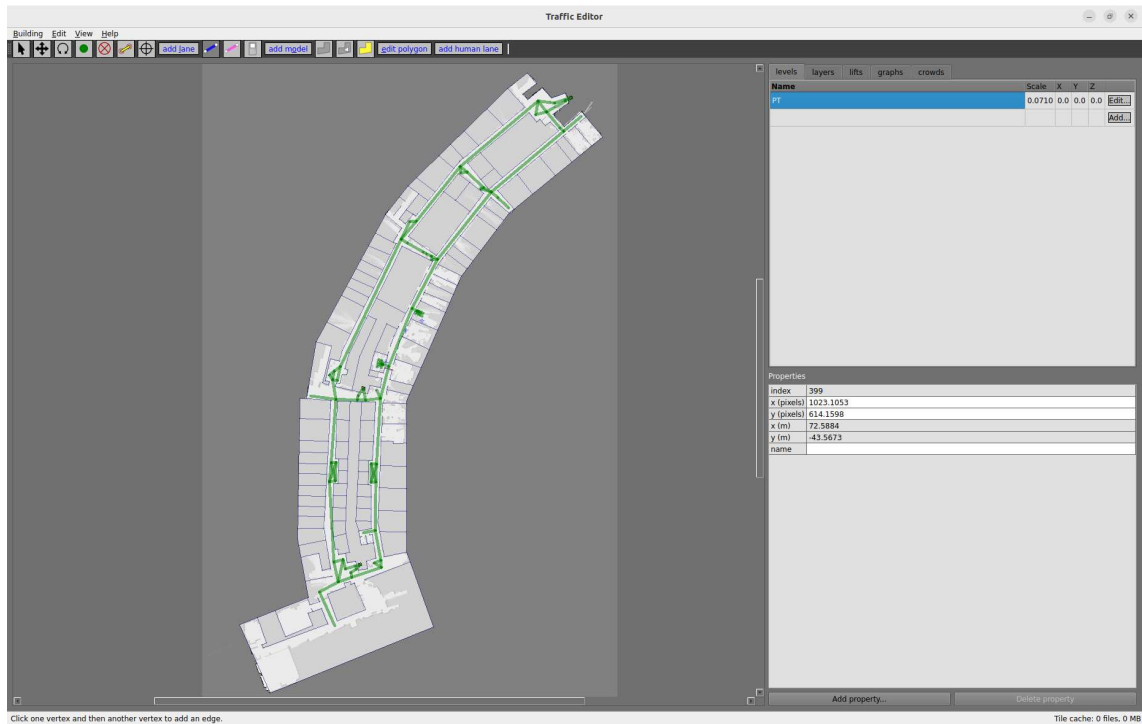


Figure 11.1: Traffic editor with Elettra's route map

The route map .yaml files for each graph can be generated using the following command:

```
ros2 run rmf_building_map_tools building_map_generator nav \
    ${building_map_path} ${output_nav_graphs_dir}
```

For the simulation, the world can be generated using the following command:

```
ros2 run rmf_building_map_tools building_map_generator gazebo \
    ${building_map_path} ${output_world_path} ${output_model_dir}
```

And the following command can be used to download the models required for the simulation world:

```
ros2 run rmf_building_map_tools building_map_model_downloader \
    ${building_map_path} -f -e ~/.gazebo/models
```

The simulation environment generated from the configuration resulted in the following world:

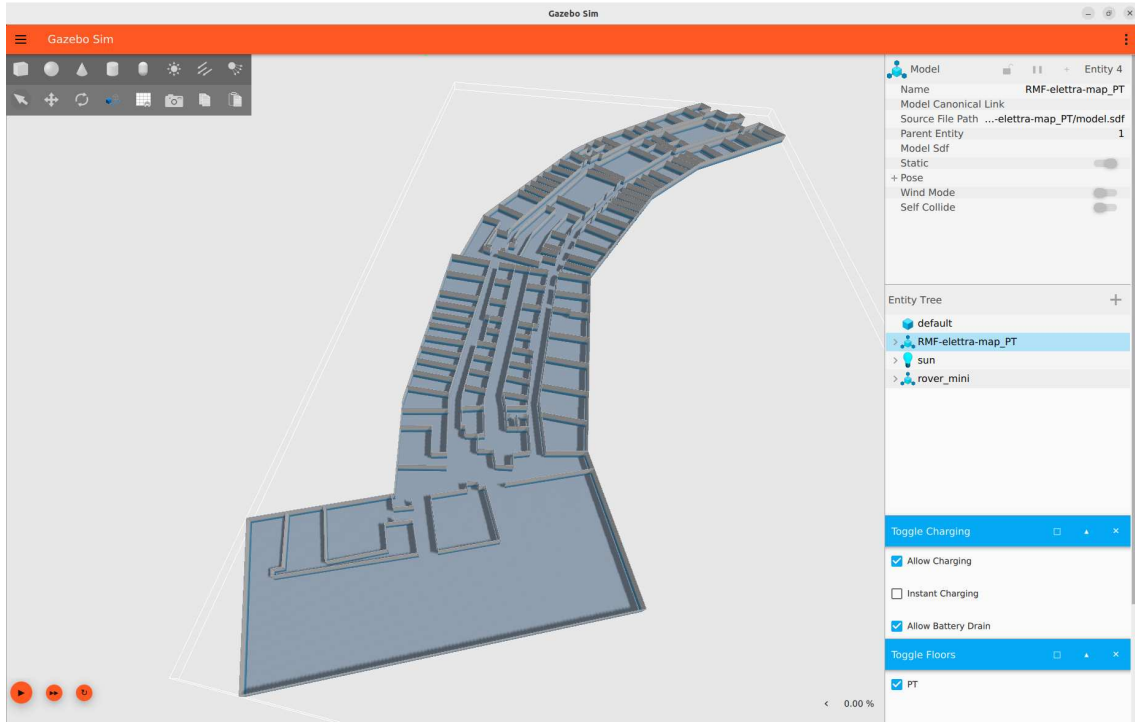


Figure 11.2: World generated using traffic editor

We can now launch the core components of RMF.

11.2 RMF core

This is the launch file inside the `rmf_server_elettra` used to start the core components of RMF:

`0_rmf_core.launch.xml`

This launch file starts the following nodes, which form the core structure of RMF:

- `rmf_traffic_schedule`: Manages traffic scheduling by controlling path structures, publishing itinerary topics and handling conflict resolution.
- `rmf_traffic_blockade`: Acts as the Traffic Blockade Moderator, managing checkpoint information.
- `building_map_server`: Publishes map data, including points of interest such as parking spots.
- `visualization.launch.xml`: Optional nodes that launches visualization tools for monitoring and controlling RMF via rviz2.
- `door_supervisor`: Publishes control requests for doors using ROS communication structures.

- **lift_supervisor**: Publishes control requests for lifts using ROS communication structures.
- **rmf_task_dispatcher**: Allow to dispatch submitted task to the best fleet/robot within RMF.
- **queue_manager**: Manages reservations for parking spots and helps prevent conflicts.

Once the core system is launched, the next step is to connect our fleets using **fleet adapters**. Open-RMF provides the Free Fleet adapter, which is compatible with ROS 2 and Nav2. The following section will detail how this adapter works and how we integrated it into our system.

11.3 Free Fleet

Free fleet⁴ is a python implementation of the `full_control` Open-RMF Fleet Adapter⁵, which serves as a general template for communicating with a fleet manager. Its primary goal is to control robots running under both ROS 1 and ROS 2. It uses **zenoh** as a communication layer between each robot and the fleet adapter. Below is an overview of how communication is managed using the Free Fleet adapter:

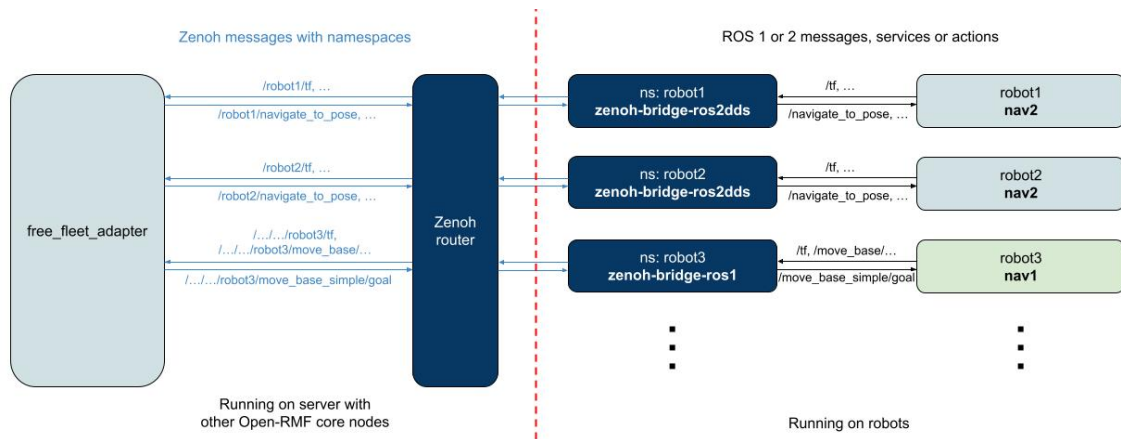


Figure 11.3: Free Fleet Adapter Communication

11.3.1 Setup fleet

To use the Free Fleet, it is necessary to create a `.yaml` configuration file for the fleet. This file describes the robots within the fleet that share the same characteristics. The configuration includes essential information such as the fleet name, velocity limit of the fleet, robot dimensions and mechanical details, power management parameters, the tasks the fleet are capable of performing, a list of all robots in the fleet with their individual names and their assigned chargers. It also need the positional correlation between the robot's map and the RMF map.

After setting up the fleet configuration, the next step is to configure Zenoh for communication.

⁴https://github.com/open-rmf/free_fleet

⁵https://github.com/open-rmf/fleet_adapter_template

11.3.2 Setup Zenoh

Zenoh⁶ is a pub/sub/query protocol that unifies data in motion, data at rest and computations. It combines traditional pub/sub with geo distributed storage, queries and computations, while maintaining high efficiency in both time and space. Zenoh allows selective topic transmission over the network, reducing bandwidth usage to specific servers.

To set up Zenoh for use with Free Fleet, the following steps are required:

- Zenoh router must be running on a server. This router can be launched using the command: (`$ zenohd`). Multiple instances of the Zenoh router can be run for redundancy.
- On each robot and on the RMF server, the standalone executable `zenoh-bridge-ros2dds` must be launched with the associated configuration file. This executable bridges all ROS 2 communications using DDS over Zenoh. The configuration of this file depend on the machine. On the server it's configured to accept all the traffic from each robot, while on the robot we need to setup:
 - The `namespace`, which must match the fleet configuration file namespace.
 - Enabling the publication of the robot's frame status topics (`./tf` and `./tf_static`) and battery information (`./battery_state`)
 - Enabling communication with action servers controlling navigation (`./navigate_to_pose`)
 - Zenoh-specific configuration details, such as the communication mode, the server address, the protocol and the port for connection endpoints (e.g., `tcp/<ip address server>:7447`)

11.3.3 Launching Free Fleet

After completing the setup, we can use the following launch file to start the Free Fleet adapter:

```
1rmf_mini_fleet_adapter.launch.xml
```

This launch file takes as parameters the fleet configuration file and the route maps that the fleet will use. Once launched, it is possible to monitor the robot topics on the RMF server.

Also this is replicated for the Jobot fleet, launching the following file

```
1rmf_jobot_fleet_adapter.launch.xml
```

⁶<https://zenoh.io/>

11.4 Web interfaces

Open-RMF also offers a web-based interface, **rmf-web**⁷, to visualize and control the system on a browser. It consists of two main components:

- api server : Based on the OpenAPI specification and implemented with the Python FastAPI library, it provides REST API methods to communicate with Open-RMF.
- dashboard : In our case, a demo dashboard that connects to the API server and allows control of the fleet.

Both components are available as Docker images and can be launched with the command:

```
docker compose -f /path/to/compose.yml up
```

Then we can connect to the web dashboard:

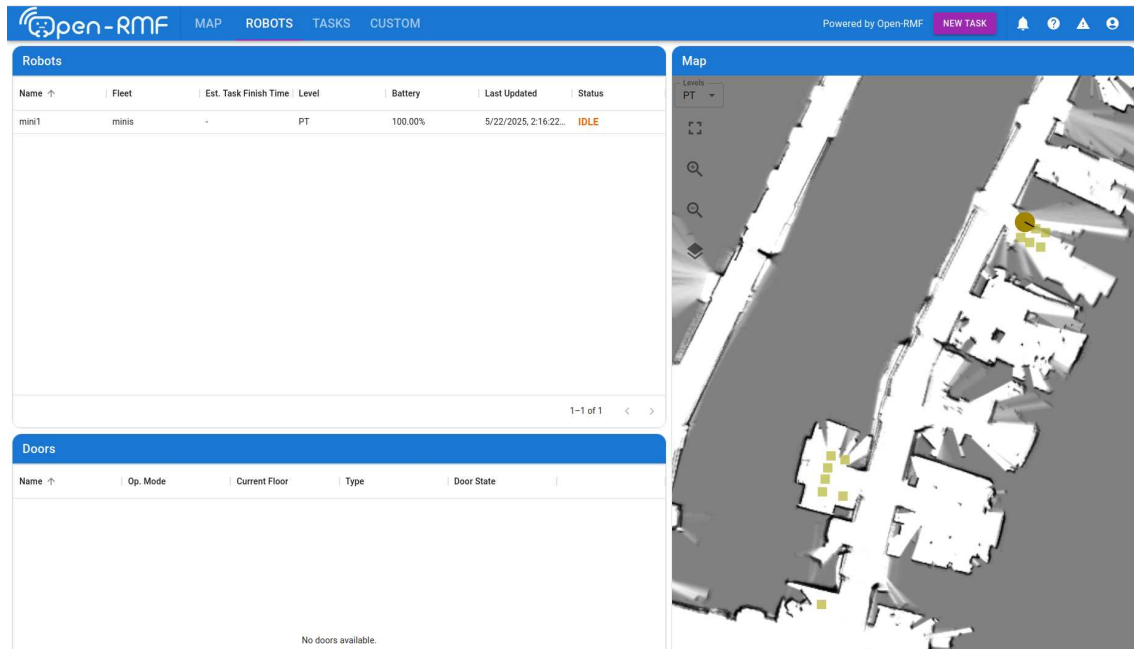


Figure 11.4: Open-RMF Web Dashboard

⁷<https://github.com/open-rmf/rmf-web>

Chapter 12

Testing and evaluation of the system

During the testing, we saw that the performance of odometry and localization needed to be improved through some parameter tuning. In the following paragraphs, I will describe the method used and the effects of the changing parameters on the output. We divided the study into two parts; the first part is based on improving the estimation of the odometry by changing the parameters of the `robot_localization` package without utilizing the correction of the AMCL node.

Then, in the second part, we will set the parameters for the AMCL node to estimate the real position of the robot with respect to the map using the newly obtained odometry data. This process was conducted using recorded sensor data collected through the rosbag package, which is described in the next section.

12.1 Testing environment using Rosbag

To allow a better comparison of the output results obtained by changing the parameters, we need a method to record the data and replay it. This solution within the ROS ecosystem is the tool Rosbag2, which allows recording and playback of communications in a ROS 2 system efficiently.

During the recording, we controlled the robots by publishing on the `/cmd_vel` topic using a joystick, following some paths that can be interesting to analyze. The path are the following:

For the Mini, we chose to use two distinct paths for the different types of configurations to be performed. For the parameters associated with the `robot_localization` package, we followed a long path with many turns:



Figure 12.1: path followed in record 001 for the mini

and this one for setting the parameters for the AMCL node because, on the right side of the door, along the corridor, the doors are closed and can reduce the robot's ability to accurately determine its position, making it crucial to fine-tune the AMCL parameters accordingly:



Figure 12.2: path followed in record 007 for the mini

For the Jobot, the path is the following one and it has similarities with the second paths used for the Mini. After passing through the hallway, the doors were closed to assess how much this would affect the AMCL.



Figure 12.3: path followed during record 001 for the jobot

Other records were made for validating the parameter choices. To record data, we use the command `ros2 bag record`. For playback, it is essential to publish to the `/clock` topic in order to avoid errors related to discrepancies in time between the recorded data and the system time: “the timestamp on the message is earlier than all the data in the transform cache.” This can be achieved with rosbag2 by using the `-clock` flag, which publishes to `/clock`. Additionally, the `use_sim_time` flag must be set to `true` for all nodes, just as it is required in simulation.

To reduce the testing time, we played the data at a rate five times higher using the flag `-r 5` for `robot_localization`, while for the AMCL we used `-r 2`; this flag increases the output rate of each topic by n times. Tests were performed with rates from 1 to 5 for multiple recordings, and the output showed that the results of `robot_localization` were similar. While the result for the AMCL was similar only with a rate of 2.

Now that we have set the testing environment, the process of setting the parameters for the `robot_localization` package is as follows.

12.2 Parameters setting for the `robot_localization`

For the analysis of the data from `robot_localization` we wanted to visualize the results of the `robot_localization` node without the influence of the AMCL node, so we set all the α parameters that define the behavior of the AMCL node to 0.0.

This allowed us to visualize in Rviz the position of the walls during the movements of the robot, allowing us to better understand its position and the type of error that the odometry exhibits.

As described before, the `robot_localization` package contains an implementation of the Extended Kalman Filter (EKF) that supports multiple sensors and allows us to select which data fields are fused with the current state estimation. It also provides its process noise covariance matrix Q to tuning it obtaining better performances.

To better understand the dynamics of our changes in the parameters, we describe in the following paragraph the theory behind it.

We want to know the state x , given a non linear system f with process noise w :

$$x_k = f(x_{k-1}) + w_{k-1} \quad (12.1)$$

and with a measurement z given by the sensor model h with noise v :

$$z_k = h(x_k) + v_k \quad (12.2)$$

Using a prediction step:

$$\hat{x}_k = f(x_{k-1}) \quad (12.3)$$

$$\hat{P}_k = F P_{k-1} F^T + Q \quad (12.4)$$

where:

- P is the estimated error covariance.
- F is the Jacobian of f .
- Q is the process noise covariance.

And the correction step:

$$K = \hat{P}_k H^T (H \hat{P}_k H^T + R)^{-1} \quad (12.5)$$

$$x_k = \hat{x}_k + K(z - H \hat{x}_k) \quad (12.6)$$

$$P_k = (I - KH) \hat{P}_k (I - KH)^T + KRK^T \quad (12.7)$$

where:

- H is the observation matrix.
- R is the measurement noise covariance.
- K is the Kalman gain.

In short, for the process noise covariance matrix Q the better the **omnidirectional motion model** of the `robot_localization` package matches the robot, the smaller these values can be. However, if we find that a given variable is slow to converge, one approach is to increase the process noise covariance diagonal value for the variable in question. This will cause the filter's predicted error to be larger, leading the filter to trust the incoming measurement more during correction.

In the next sections we describe our procedure to tune these parameters for the Mini and for the Jobot.

12.2.1 Setting up the EKF of the mini

The data available for the Mini are given by the mounted IMU and its estimated odometry.

Using this data, and without any tuning, we obtain:

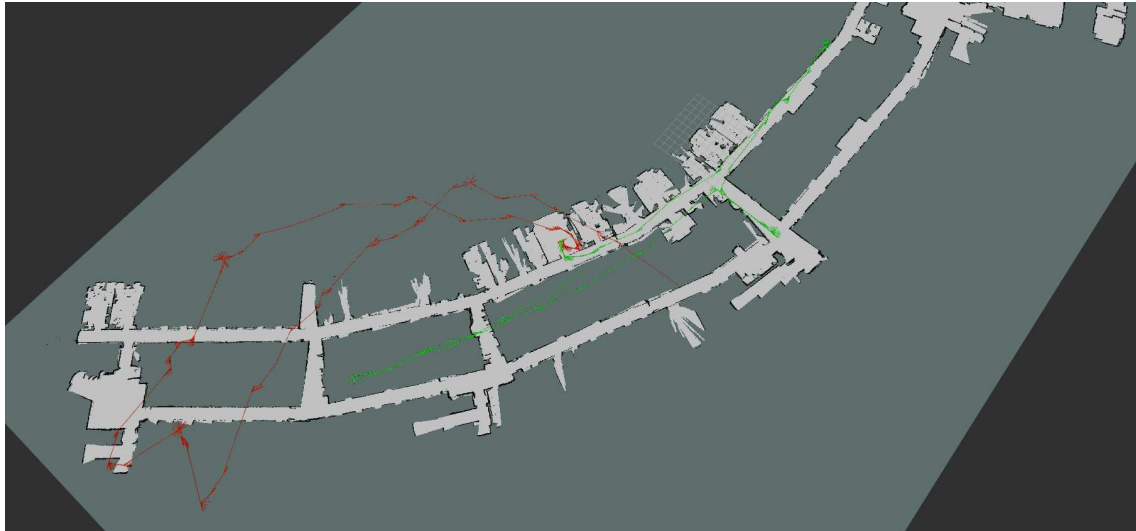


Figure 12.4: Mini odometry given by the robot (red) and odometry given by the `robot_localization` package (green)

The picture show the magnitude of the angular error for the Mini.

To compensate for this error, we analyze what data our sensor are able to provide:
IMU (topic `/imu/data` with a output frequency of 100 Hz)

- orientation : cannot be used for a random error from the sensor as visible in this picture.

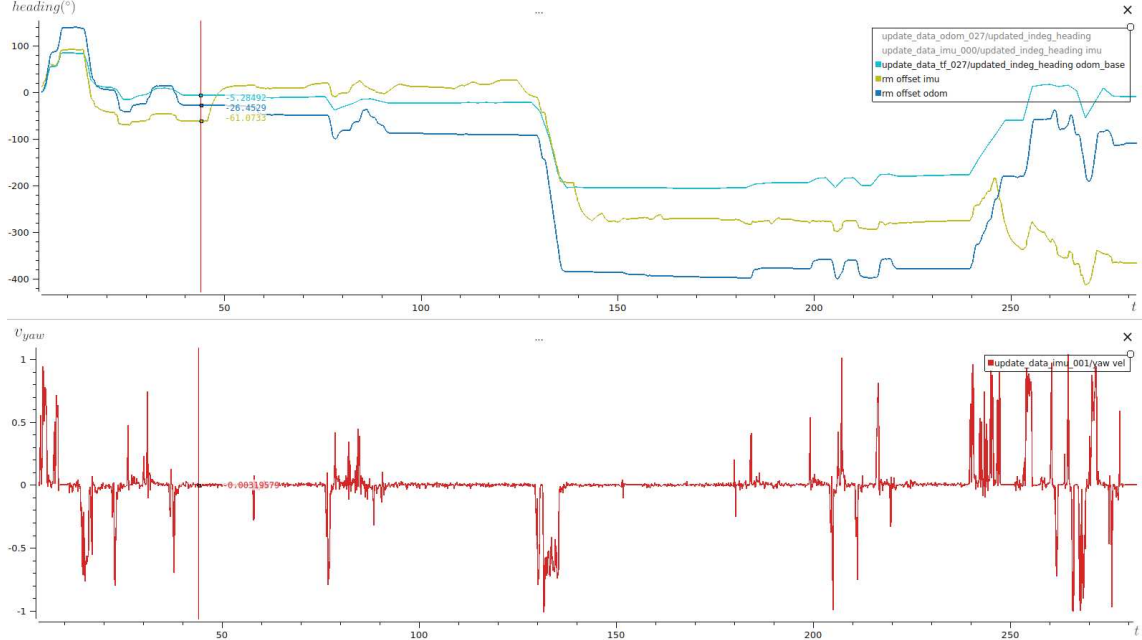


Figure 12.5: Heading data from the `robot_localization` (light blue), IMU orientation (green) and odometry from the robot (blue) versus the v_{yaw} measured by the gyroscope of the IMU

- gyroscopic : we use his data about v_{yaw}
- acceleration : add a constant increment of the position and incremented the error, so it was not used.

odometry (topic `/odometry/wheels` with a output frequency of 15 Hz)

- pose : it's not correct and add a lot of error to the estimation.
- velocity : it's also the input given to the robot, providing direct information about velocity, so we take v_x and v_y from the data. We tried to use v_{yaw} , but it introduced a lot of error in the estimation.

After choosing the input data to use, we tried changing the values of the process noise covariance Q , obtaining these results12.1. This test was performed using record 001, where the start position is inside the office and the end is outside the door of the office (a distance of about 4 m) with a total length of about 187 m:

v_x (-)	v_y (-)	v_{yaw} (-)	data name	diff start end (m)
0.001	0.001	0.001	000	12.36
0.01	0.01	0.01	001	5.88
0.1	0.1	0.1	002	5.10
10.0	10.0	10.0	003	4.37
50.0	50.0	50.0	004	4.31
0.001	0.001	0.1	005	4.83
0.001	0.001	10.0	006	4.33
0.001	0.001	50.0	007	4.27
0.001	0.1	0.001	008	5.88
0.001	10.0	0.001	009	5.88
0.001	50.0	0.001	010	5.88
0.1	0.001	0.001	011	5.94
10.0	0.001	0.001	012	5.94
50.0	0.001	0.001	013	5.97

Table 12.1: robot_localization results for the Mini

Some of the results:

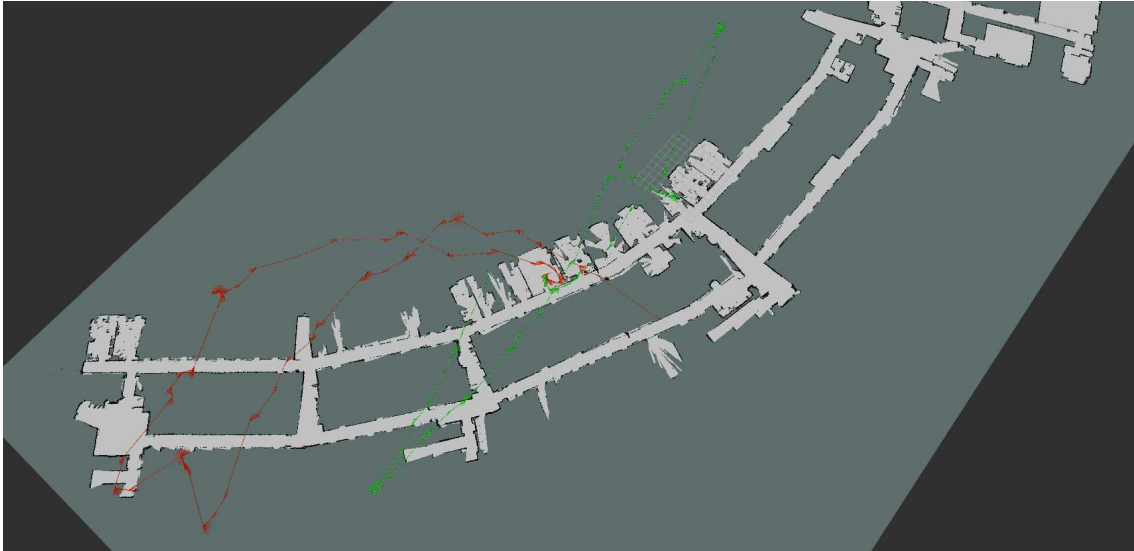


Figure 12.6: Results from data 005

All the test are displayed in the next graph:

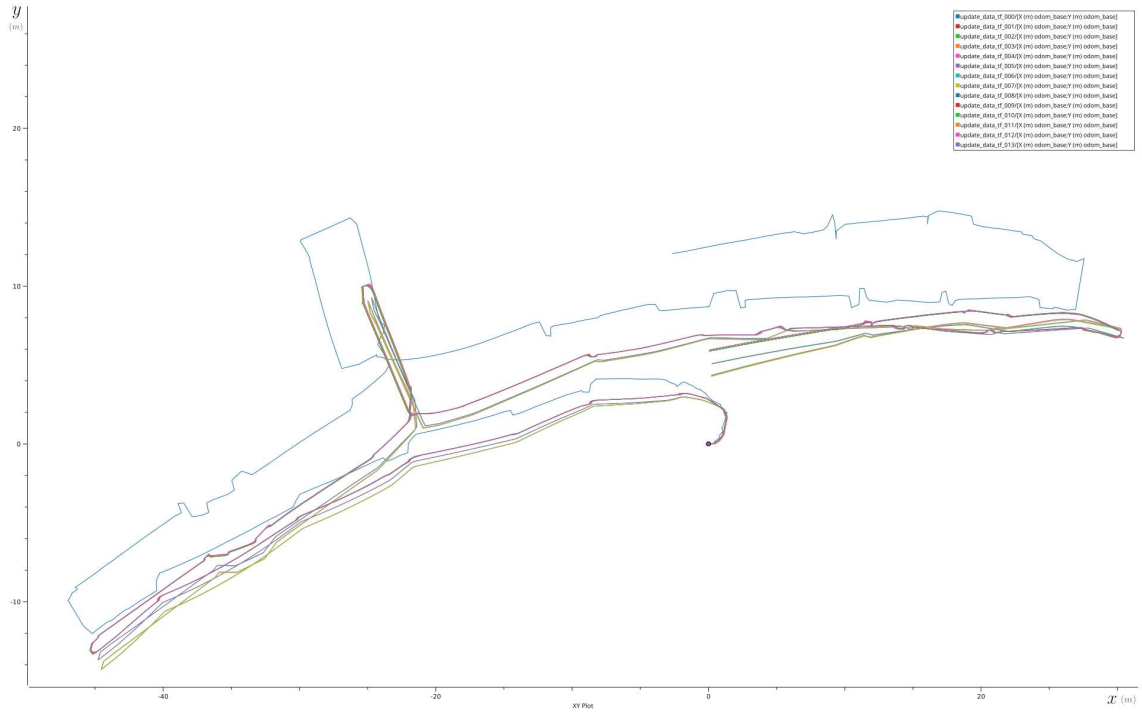


Figure 12.7: Results changing parameters of the process noise covariance using the record 001

We can see that the best results were achieved by incrementing the value of the process covariance matrix for v_{yaw} , while changing the other inputs did not affect the results. More tests were performed with larger and smaller values of those variables, but they did not change the outcome. So, the best result was obtained whit a value for v_{yaw} of 50.0 while for the other two keep at 0.1 and gives us this result:

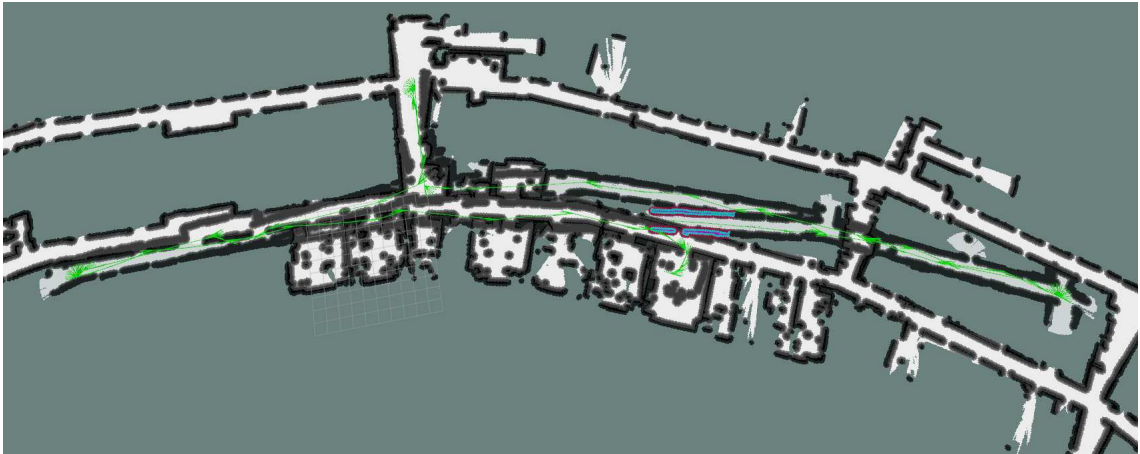


Figure 12.8: Best result for the Mini with the information of walls position

As shown in Figure 12.8, examining the correspondence between the walls of the map and the walls recorded during navigation indicates that the estimated position is incorrect. However, adjusting the values in the process noise matrix did not improve this estimation. This is a correction that the AMCL node is intended to address.

12.2.2 Setting up the EKF of the Jobot

A similar procedure to that used for the Mini was performed for the Jobot. We discovered that in both recordings, the robot, while moving through a fire door, makes a "jump" visible in the odometry. This may be caused by a small step needed to pass through the fire door.

From some initial tests, it is evident that the estimation of the length of the path followed by the robot is correct, so we focused our attention on the observable error in rotation. The data of measure 000 was performed with only the *yaw* of the odometry of the robot:

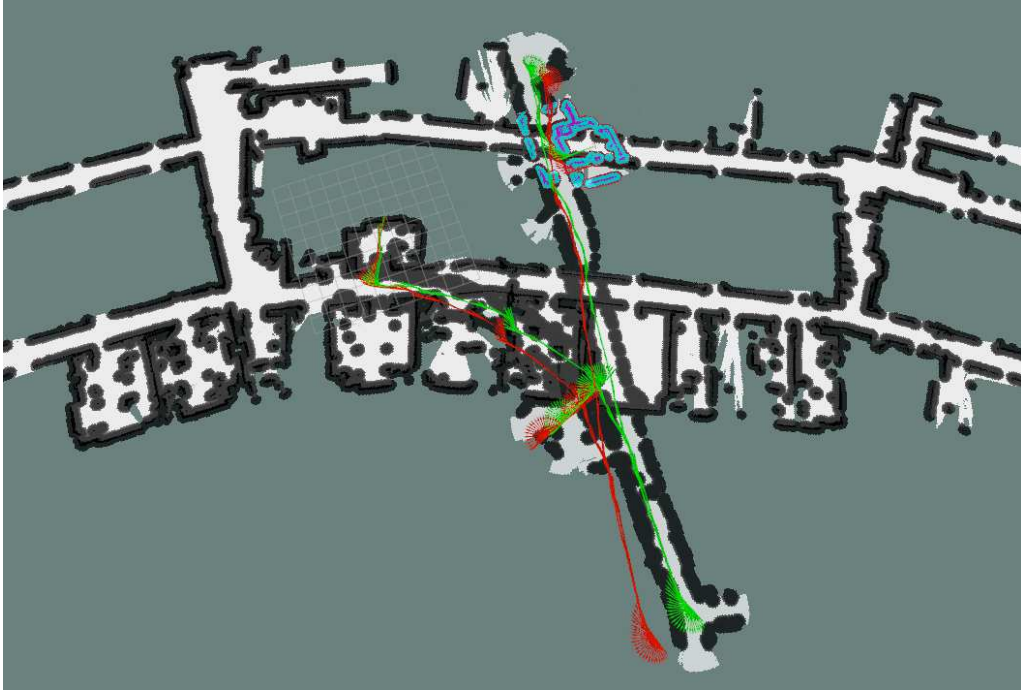


Figure 12.9: Estimation obtained with the *yaw* of the Jobot's odometry (green) with the estimation from the robot (red)

Having seen the results and tested some changes to the process noise covariance did not affect the outcomes, it was decided to remove this data and evaluate the results using v_{yaw} from only the IMU (data 001):

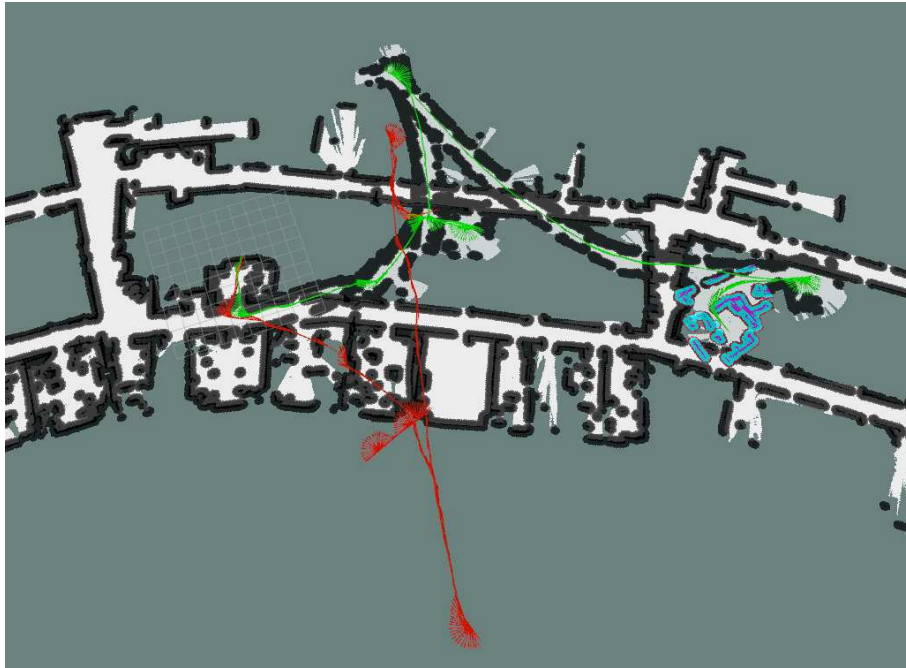


Figure 12.10: Estimation obtained with only the v_{yaw} of the Jobot's IMU (green) with the estimation from the robot (red)

It overestimates the rotation, so we tried summing v_{yaw} from both the IMU and the odometry provided by the Jobot (data 002):



Figure 12.11: Estimation obtained with both the v_{yaw} (green) with the estimation from the robot (red)

Since this was the best outcome, we decided to use them as input and change the values of Q for the v_{yaw} because the error in translation is very low. All the results are:

v_{yaw} (-)	record name	diff start end (m)
-	000	14.28
-	001	32.67
0.1	002	13.88
10.0	003	13.26
100.0	004	8.74
200.0	007	4.31
300.0	008	1.83
400.0	009	3.50
320.0	012	1.63

Table 12.2: Result tuning parameters process noise covariance for Jobot

We obtained the best result with a value of 320 for the process noise covariance for the value of v_{yaw} .

The best outcome is visible in the following figure:



Figure 12.12: Best result obtained for Jobot (green) with the estimation from the robot (red)

The plotted data are:

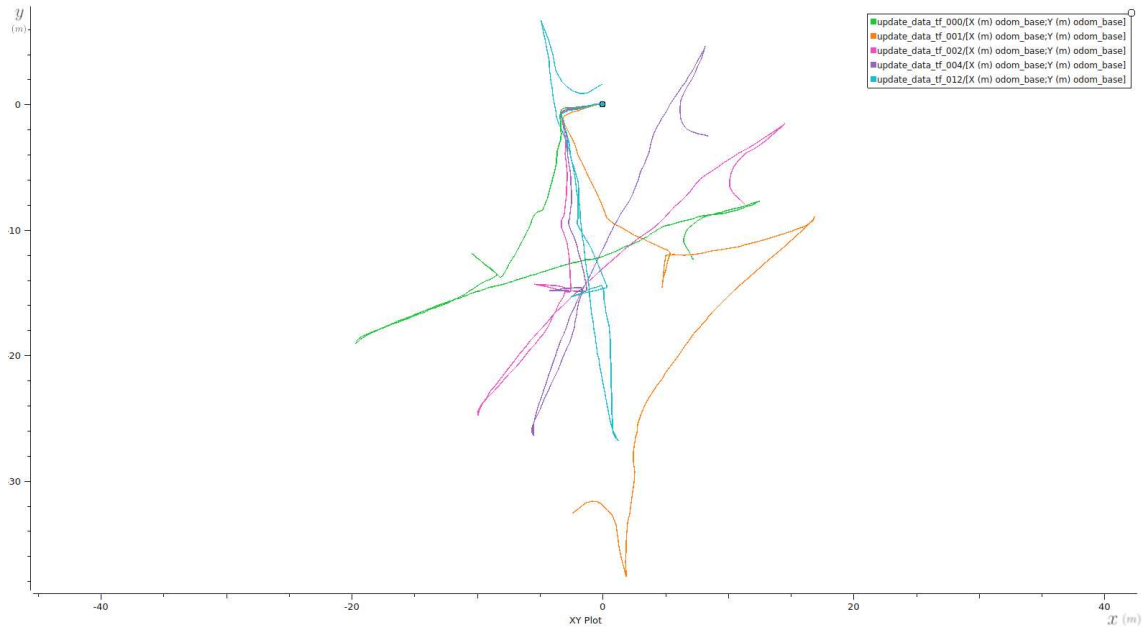


Figure 12.13: Results from the tuning of the process noise covariance for the Jobot plotted

12.3 Parameter tuning for the AMCL

Calibrated the process noise matrix Q for both robots. Now, we need to estimate the correct position concerning the map (global localization), which is achieved using the AMCL node as described earlier. This also compensates for all the errors caused by incorrect odometry estimation. In this section, we will adjust the α parameters and observe the results. A description of these parameters is as follows:

- $\alpha1$: Expected process noise in odometry's rotation estimate from rotation.
- $\alpha2$: Expected process noise in odometry's rotation estimate from translation.
- $\alpha3$: Expected process noise in odometry's translation estimate from translation.
- $\alpha4$: Expected process noise in odometry's translation estimate from rotation.

12.3.1 Setting up the AMCL of the mini

These tests were performed on record 007 and the starting position is about 0.5m from the ending position. This choice was made because it was conducted in a challenging situation that demonstrates the difficulties an AMR may encounter. In the first half of the path, the robot encounters enough features that can be used by the AMCL node to retrieve the correct position relative to the map (the doors in that part of the corridor are open, providing sharp edges for the laser scanner). However, in the second half, there are not many environmental details (the doors of the other half of the corridor are closed), and the position estimation may fail.

In the following figure, the odometry estimation with the α parameters set to 0 for record 007 is shown.

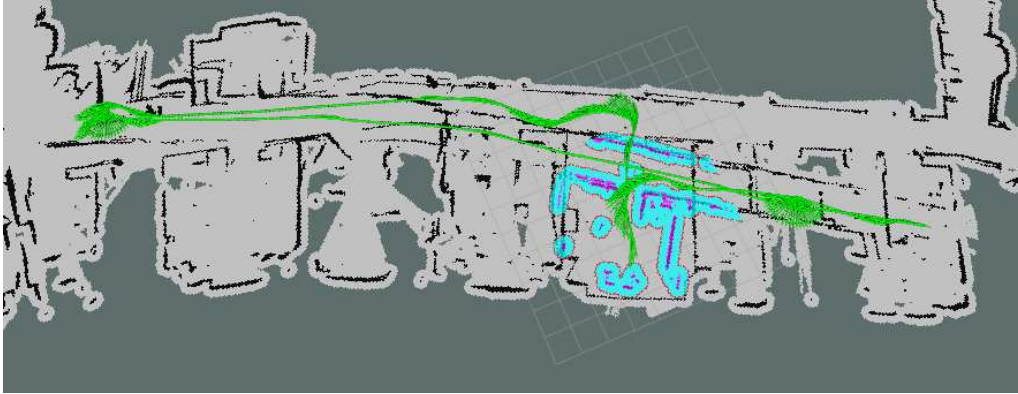


Figure 12.14: Best result with the Mini using the `robot_localization` for the record 007

Then, we tested changing only one parameter at a time, increasing its effect on the estimation while paying closer attention to the parameters that most effectively correct the estimation made in the `robot_localization` node. This is necessary to compensate for errors during rotation and also errors in the traveled position. This can be compensated using the parameters: $\alpha1$ and $\alpha3$.

The results are shown below, with the first row displaying the data from odometry provided by the localization node without correction from the AMCL node.

$\alpha1$ (-)	$\alpha2$ (-)	$\alpha3$ (-)	$\alpha4$ (-)	distance (m)	data name
0.0	0.0	0.0	0.0	1.92	-
0.001	0.0	0.0	0.0	1.32	000
0.01	0.0	0.0	0.0	0.60	001
0.1	0.0	0.0	0.0	0.64	002
0.0	0.001	0.0	0.0	0.58	003
0.0	0.1	0.0	0.0	0.51	004
0.0	0.0	0.001	0.0	2.01	005
0.0	0.0	0.01	0.0	2.27	006
0.0	0.0	0.1	0.0	2.32	007
0.0	0.0	0.0	0.1	3.08	008
0.01	0.01	0.01	0.0001	0.53	009
0.01	0.01	0.1	0.0001	0.69	010

Table 12.3: result planner first request

Changing $\alpha1$ and $\alpha2$ fixes the rotation error, but we observe significant oscillation in the position correction if we increase them too much; this was seen in data 004. In the end, we decided to keep both parameters around 0.01. As a result, we obtained this picture of the odometry, which shows that the rotation was corrected.

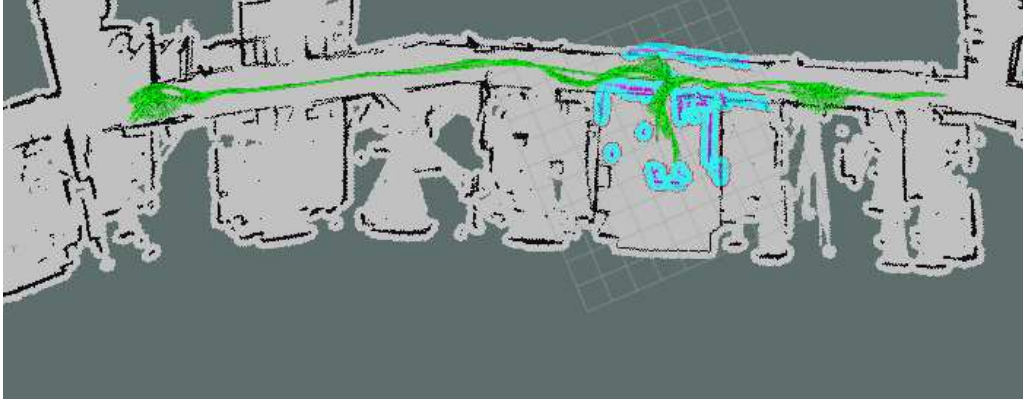


Figure 12.15: Result performed by AMCL without length correction

This allowed us to observe the corrections made for the heading. It is evident that the error increases as the robot moves and we can see that we performed a good correction in rotation using the `robot_localization` package but not in translation.

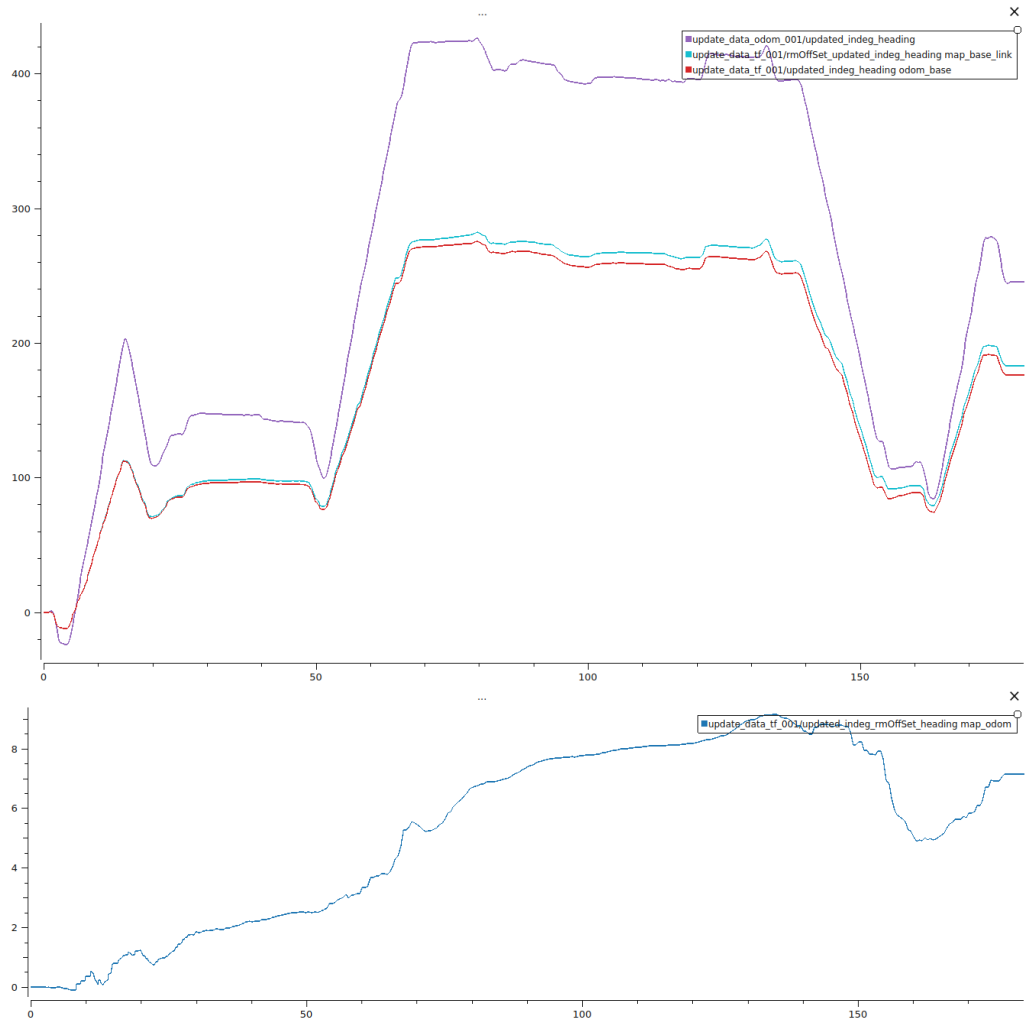


Figure 12.16: On top heading values returned by the system; down the heading correction for the Mini performed by AMCL

The table shows that without adding the correction for rotation, the translation correction parameter alone does not solve the problem. During testing (data 009), it was possible to see that the combination of these parameters allowed for a correction of the position on the left side of the path (the one with more map detail). This did not completely resolve the issues on the right side (with less map detail), but it was able to make some corrections. The final solution (data 010) is considered the best, effectively addressing the position error during movement. The parameter was kept high to accommodate worse scenarios, such as longer hallways.



Figure 12.17: Best result obtained by AMCL on the Mini on record 007

And this is the data plotted.

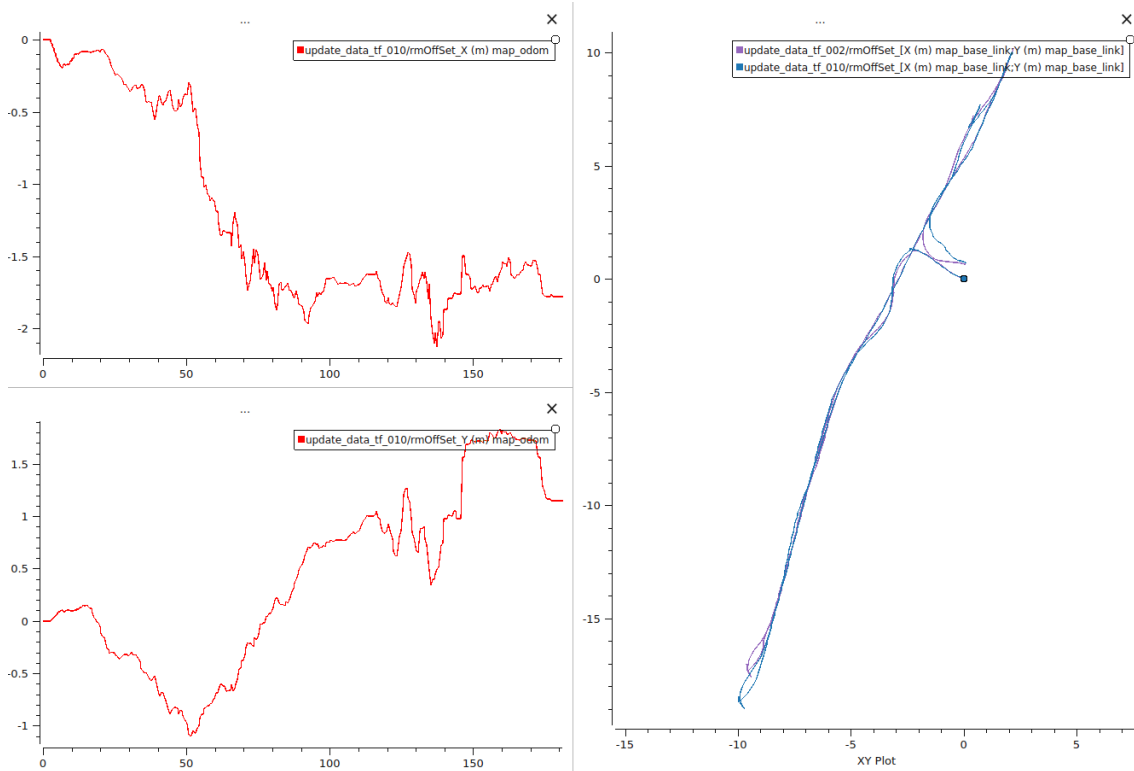


Figure 12.18: On the left the correction in position performed by AMCL; on the right the path corrected by AMCL

It is evident that to the left of the starting point, the translation error is fully compensated. In contrast, to the right, the performance is poorer due to a lack of features, as many doors are closed along that path.

12.3.2 Setting up the AMCL for the Jobot

As mentioned before, in both records, when the robot passed the fire door, it "jumped," creating an error in the estimation. We decided to keep the data and observe how the AMCL package recovers from this jump. For record 001, which we will use in our test, the difference in distance from the starting position to the ending position is around 0.6 m. As shown in figure 12.12, the error that needs correcting is the angular position generated during rotation around the axis. Therefore, as previously described, we need to adjust the parameter $\alpha1$, which relates to the expected process noise in the odometry's rotation estimate. For this test, we will increase that value and observe how the output changes.

$\alpha1$ (-)	$\alpha2$ (-)	$\alpha3$ (-)	$\alpha4$ (-)	distance (m)	data name
0.0	0.0	0.0	0.0	1.63	-
0.001	0.0001	0.0001	0.0001	1.58	000
0.01	0.0001	0.0001	0.0001	2.16	001
0.01	0.001	0.0001	0.0001	0.71	002
0.01	0.001	0.001	0.0001	0.87	003
0.01	0.001	0.001	0.0001	0.74	004
0.01	0.001	0.001	0.001	0.59	005

Table 12.4: Result AMCL Jobot

In the first row, the output of the estimation in odometry without correction is reported. With data 000 and 001, we observe that the robot did not recover from the jump, resulting in an increasing error.

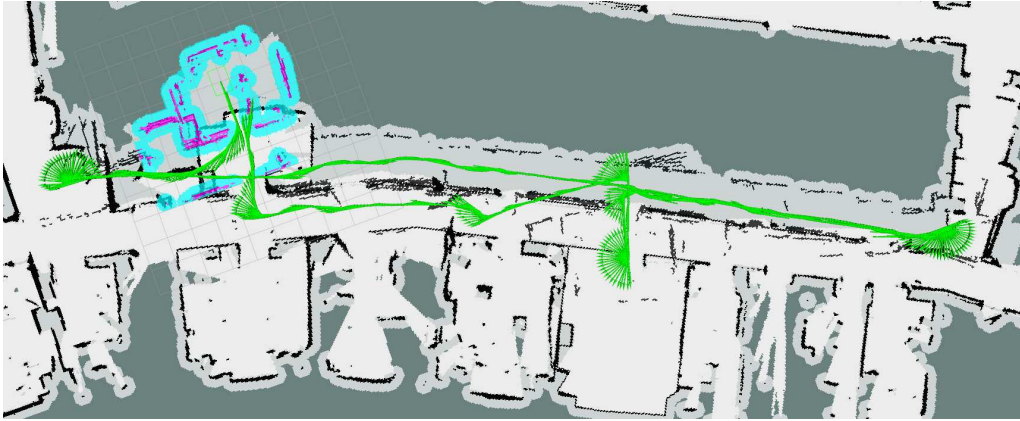


Figure 12.19: Figure of data 001 obtained by Jobot

Because we didn't increase the correction for rotation during translation $\alpha2$, and since we moved straight after the jump, AMCL did not fix the rotation error. So, we increased it, achieving good results. To obtain even better outcomes, we added a bit more correction with $\alpha3$ and $\alpha4$. Considering that our test path was short and there could be additional errors in translation, we also observed an increase in estimation performance.

The plot of the data for the best results is as follows:

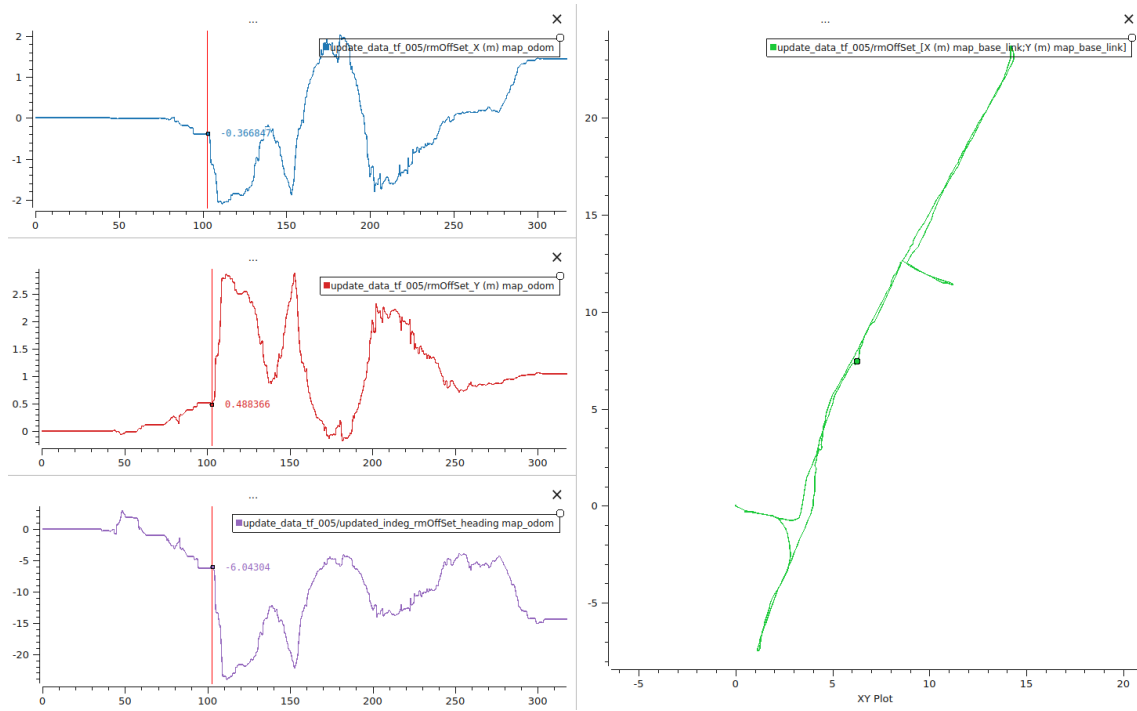


Figure 12.20: Plots of the best result for the Jobot with record 001; on the left the correction along x , y and for the $heading$; on the right the path followed by the robot

At the vertical red line, it is visible that the small step generated a very high error in all three states: x , y and yaw , but AMCL managed to correct it perfectly. It was also tested on record 002, which was created with the intention of generating errors in the estimation due to an environment lacking a sufficient number of features. In fact, it was decided to move the robot along the corridor and, before coming back, to close the doors of some offices. Since the odometry estimated by the robot is good and the corrections in the AMCL are minimal, this did not create a problem. Here is a picture of the result:

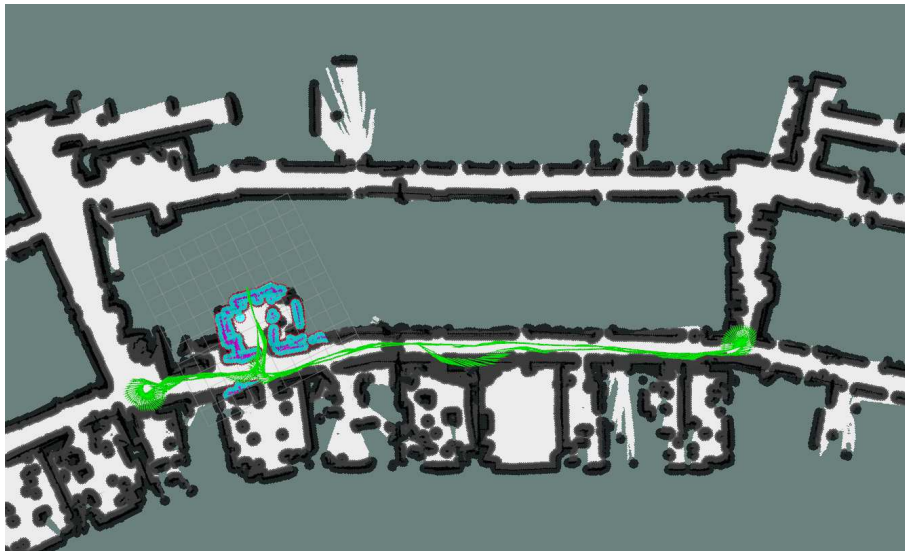


Figure 12.21: Result of the Jobot with record 002

12.4 Navigation problems

12.4.1 In simulation

Now we tested the AMRs in simulation and discovered that for the Jobot, during sharp curves, as shown in figure 12.22 below, the AMR stopped right before the turn, rotated in place, and then proceeded to hit the wall, eventually stopping.

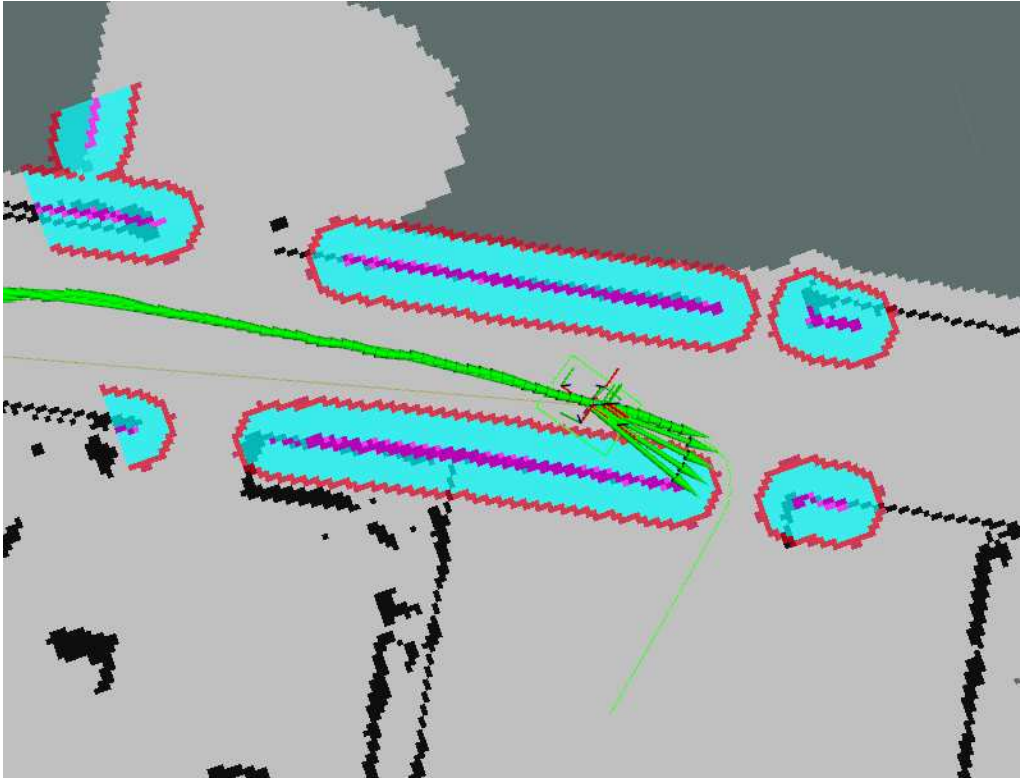


Figure 12.22: `problem_local_planner_jobot`

This issue is caused by the local planner failing to generate an effective command to move the AMR. It takes the global plan and the local costmap and produces command velocities to direct the AMR toward the goal. One solution was to change the local planner from the default Dynamic Window Approach DWB successor of the DWA, based on the sampling of possible velocities and selecting the one with the best score, to the Model Predictive Path Integral Controller MPPI[22], which uses a sampling-based approach to select optimal trajectories, optimizing across successive iterations. This approach has shown better performance[15] and works without any configuration. The other solution was to tune the DWB to avoid this behavior.

12.4.2 In reality

As described earlier, the Mini has a high error in position estimation due to the localization and the lack of sufficient feature description of the map using the scan. If we ask the robot to move to a location and the AMCL doesn't manage to correct the position in time, the robot will respond that no path was found. This happens because the position of the door in the global costmap is not aligned with the position of the door in the local costmap, resulting in the path being blocked for the global

planner, as shown in picture 12.23.

One solution could be to rotate beforehand, allowing the AMCL to correct the error between the odometry frame and the map frame by adjusting the parameter that regulates the error caused by rotation. This can be implemented in openRMF by adding a vertex before the door to help correct the translation error, enabling the generation of a path to enter through the door. Another solution is to improve the odometry estimation of the Mini by using absolute reference markers visible to a camera[2], allowing the localization node to have periodic absolute corrections, similar to GPS. This enhancement can also facilitate corrections and movement outside the facility, allowing navigation between buildings to correct its absolute position.

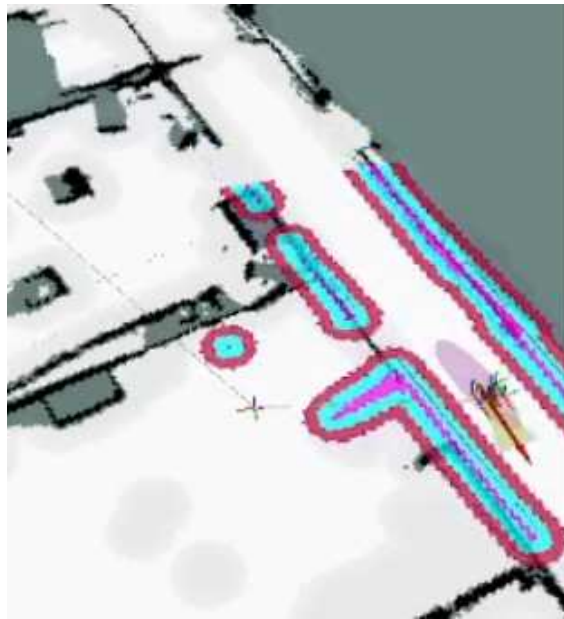


Figure 12.23: Error when also AMCL fail

Additionally, to start all nodes in the AMRs, systemd services are added.

12.5 Other problems discovered during testing

After some testing, these are the problems we found:

- The testing revealed some design issues with the rover. When operating in an office environment, the robot's laser sensor cannot detect certain parts of obstacles, as shown in the image below.



Figure 12.24: Laser position error

One possible solution is to reduce the free space from the occupancy map. However, the dimensions of the chair legs are quite long, significantly reducing the available space to pass through doors, which the planner currently prevents. Another option is to relocate the LIDAR sensor underneath the robot. This would require a low-profile LIDAR and would create blind spots by the wheels if no additional sensors are added on top. A cheap solution is to add some ultrasonic sensor with an high field of view to catch these object that the LIDAR doesn't see. Alternatively, adding a depth camera could provide a 3D occupancy grid, which can then be processed to generate a 2D costmap.

- The ROS 2 infrastructure allows all devices within the same subnet to connect through multicasting. To avoid multiple nodes publishing to the same topic and creating errors in communication, several solutions are available:
 - some depending from the middleware used (cycloneDDS, FastRTS),
 - assign each robot with a **namespace**,
 - associated each robot with a unique **ROS_DOMAIN_ID**.

The solution that best applies to us and is adopted in the Free Fleet package is the use of the **ROS_DOMAIN_ID**. This was also implemented in the simulation. Consequently, all important topics are shared through the Zenoh plugin, as described earlier.

- The D-star algorithm does not perform effectively within the ROS 2 planner ecosystem because the map update procedure does not allow the planner to detect obstacles, this implementation has been included in the roadmap.
- An increment in the security is the setting up collision monitor for improve safety reason. This node use the information of some sensor and reduce the velocity sended to the robot in function of near obstacle.

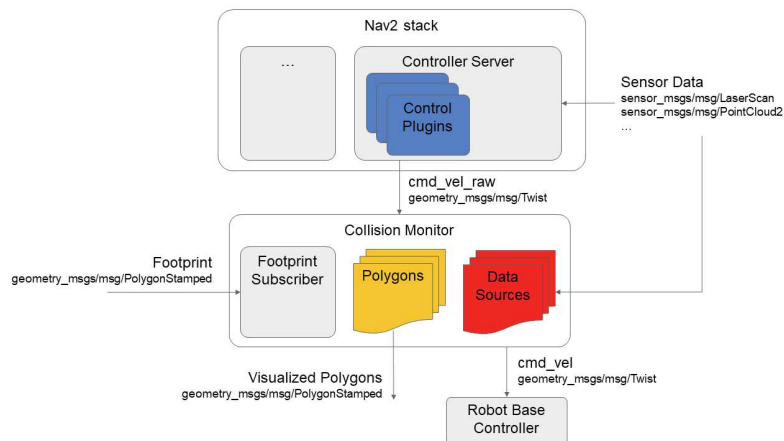


Figure 12.25: Collision monitor diagram

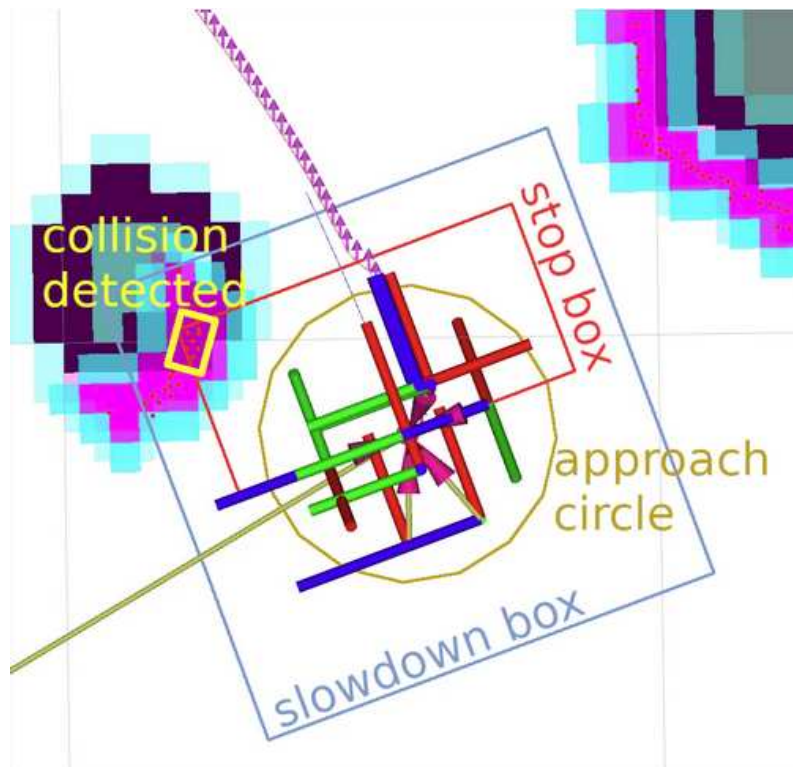


Figure 12.26: polygon display

- Additionally, we discovered misalignment between the planimetry image and the Cartographer map, as shown in the image below. It appears that at each major rotation, the system overestimates the robot's rotation angle during mapping.

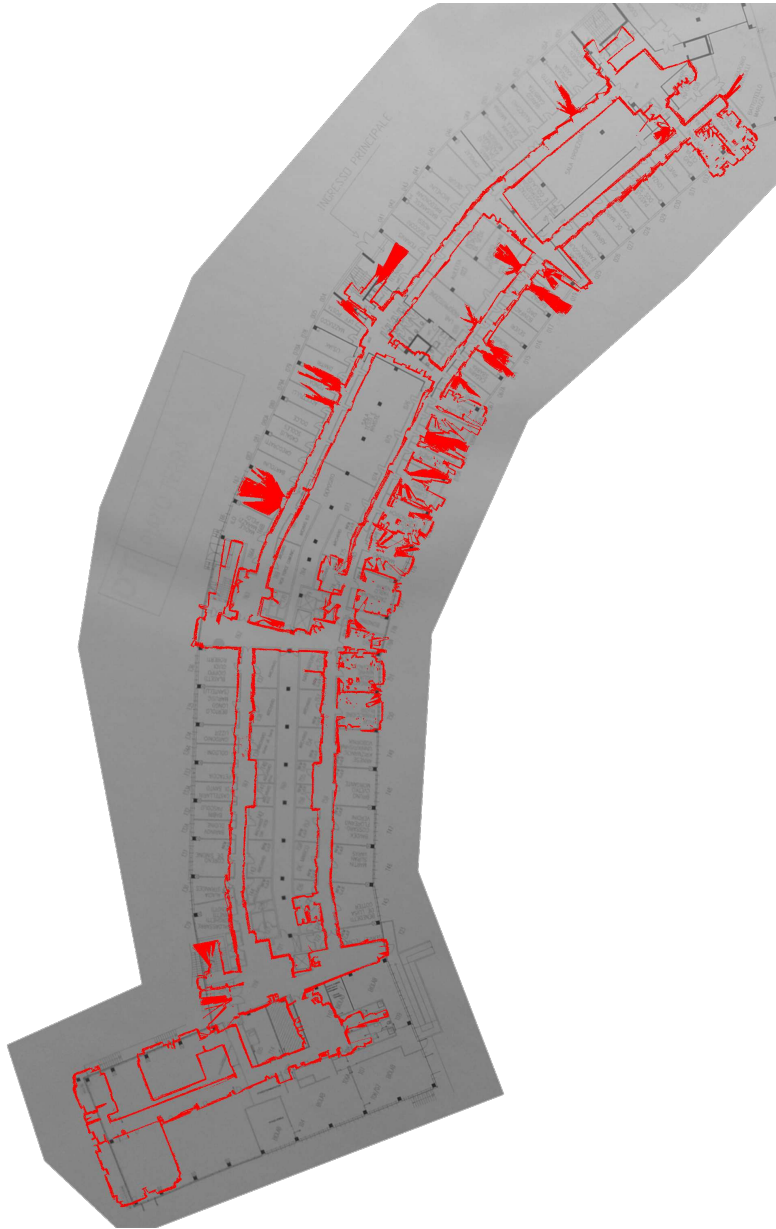


Figure 12.27: Overlap Cartographer Map and photo of floor plan

Part IV

Results and Analysis

Chapter 13

Presentation of results from testing and evaluation

This part is divided based on the things that have been completed:

- porting the planner DStarGlobalPlanner from ROS 1 to ROS 2.
- "building" two Autonomous Mobile Robots (AMR):
 - from scratch and using the Rover Robotics Mini platform.
 - proving the portability of the code structure used for the Jobot updating it to ROS 2.
- implement the Open-RMF middleware using these robots.

13.1 Porting the planner

We have successfully ported the code to ROS 2 and used it for navigation. This is an output plan generated by the planner. More information about the functionality of the parameters is described in this paper.[19].



Figure 13.1: results obtained by the Dstar planner

In the end, the logical transformation from the `move_base` logic in ROS 1 to the Planner Server used in ROS 2 for the global planner is based on the application of more advanced functionalities of the programming languages used. We also discovered that several improvements need to be made to enhance the performance of the planner within ROS 2.

13.2 Build 2 AMR in ROS 2

In the end, both robots were added, both in simulation to test the capabilities of OpenRMF and in real-time to apply our results. All the capabilities of Nav2 were utilized, along with a study of its parameters to ensure optimal control. The following pictures show the robots in simulation.

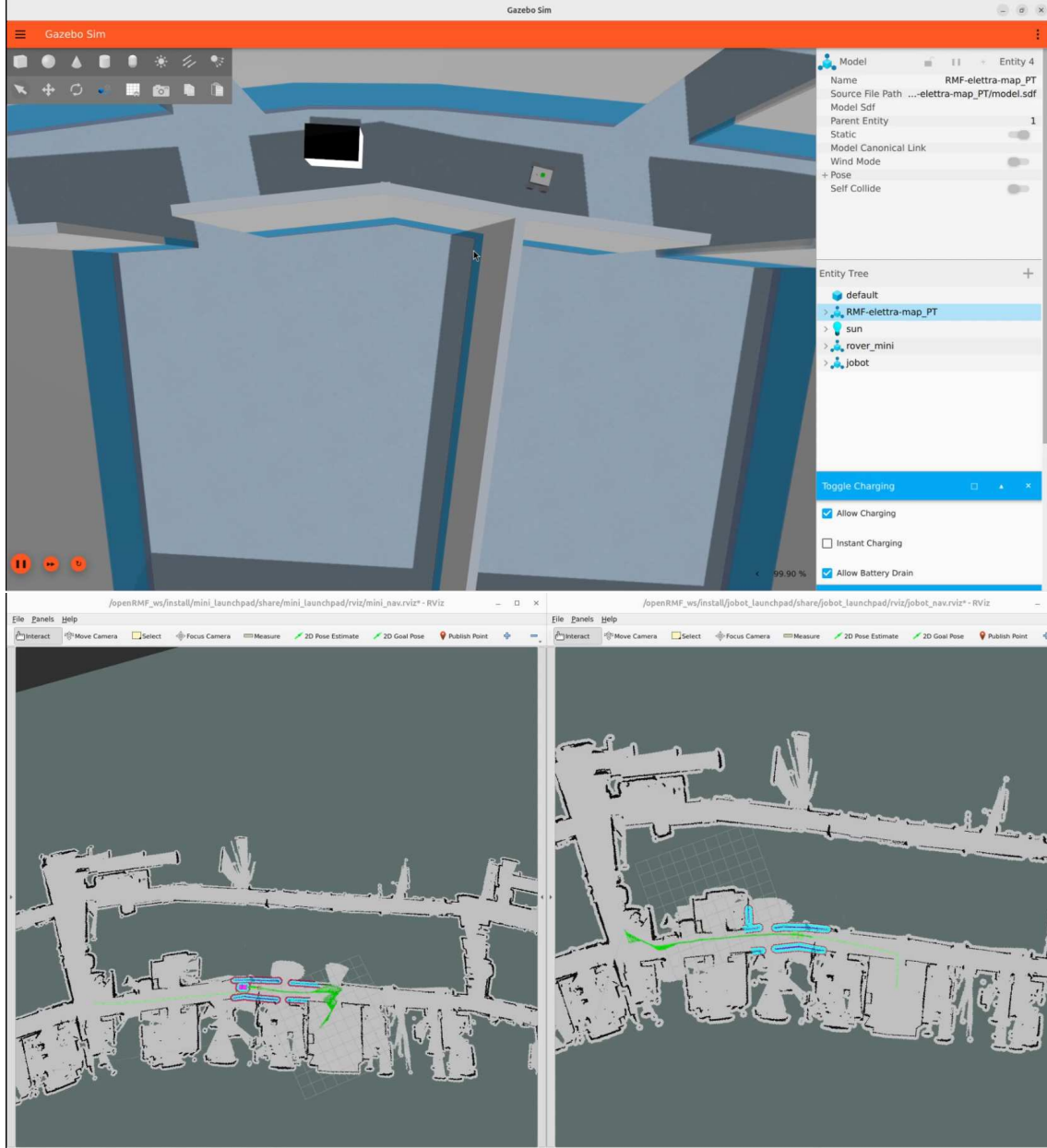


Figure 13.2: both robots simulated

13.2.1 In particular for the Mini

We found issues with his position estimation, which we resolved by tuning the parameters of the AMCL accordingly. We achieved the best performance with the following values for the process covariance matrix: v_{yaw} of 10.0 and for AMCL: α_1 0.01, α_2 0.01, α_3 0.1 and α_4 0.0001

13.3 In particular for the jobot

We converted all the code used to control the robot from ROS 1 to ROS 2, demonstrating the portability of that code structure. We added the Lifecycle to be implemented as part of the boot sequence of the robot with Nav2 and its sensors. We corrected the estimation in rotation by combining the velocity estimates from the robot and the gyroscope and with the AMCL we corrected the remaining error. v_{yaw} of 320.0 and for AMCL: α_1 0.01, α_2 0.001, α_3 0.001 and α_4 0.001

13.4 Open-RMF navigation result

These are some images of the navigation of the 2 robots

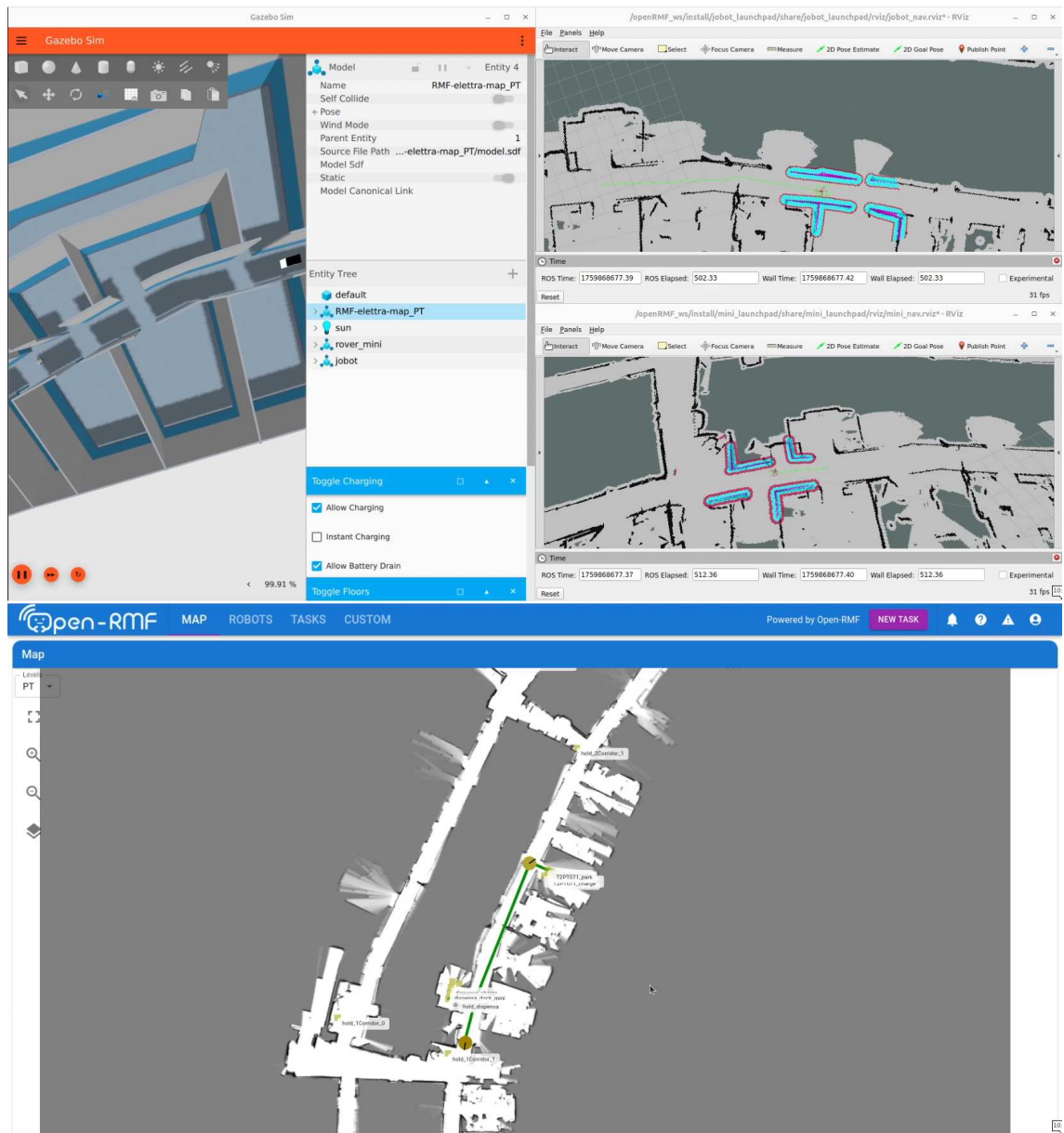


Figure 13.3: images navigation of the 2 robots with open-rmf in simulated environment

The results are promising, and this is a solution that can be implemented in our

workspace. We tested the patrol task, and the robots moved through the building, avoiding traffic jams and finding the best paths.

Chapter 14

Comparative analysis on the port system

In this section, we evaluate the time required by the planner to find a path, comparing it with the default planner.

14.1 Planner calculation time comparison

In this section, we show the time required for the global planner node to calculate different path, we compared the performance between the DStar global planner and the NavfnPlanner over 3 path: from the office to the storage room (used as base in the setting up the parameters), from the office to the entrance of the building (fig: 14.1a,14.2a)and from the entrance to the our CAD designers location(fig:14.1b,14.2b). The results were obtained in a jazzy docker image with a Intel i5-1345U CPU and adding the flag:

`-cmake-args -DCMAKE_BUILD_TYPE=Release`
to the call of `colcon build`.

Start->End	ROS 2 D-start (s)	ROS 2 NavfnPlanner (s)
office -> storage room	1.00	0.020
office -> entrance	5.47	0.150
entrance -> CAD designers	5.36	0.130

Table 14.1: result planner first request

After the first calculated path:

Start->End	ROS 2 D-start (s)	ROS 2 NavfnPlanner (s)
office -> storage room	0.004	0.022
office -> entrance	0.009	0.156
entrance -> CAD designers	0.011	0.135

Table 14.2: result planner first request

There are some results generated using the planner Dstar:



(a) office -> entrance



(b) entrance -> CAD designers

Figure 14.1: Path plans generated by Dstar planner

and these are some generated by the NavfnPlanner:



(a) office -> entrance



(b) entrance -> CAD designers

Figure 14.2: Path plans generated by NavfnPlanner planner

the result showed us:

- time to calculate the first plan using Dstar is very higher than the NavfnPlanner.
- after the first calculated path the algorithm return the same path so this explain this fast response.

We see that optimization must be made to the code of the ROS 2 global planner to unveil the potential of the D* algorithm since is not full designed to work together with ROS.

Part V

Conclusion and Future Work

Through this work, we conclude the advantages that ROS 2 brings to the industrial environment and the potential offered by OpenRMF. In this chapter, we describe all the advantages we have discovered.

Chapter 15

Conclusion

With this work, we demonstrate the potential of ROS 2 for creating industrial robotics solutions.

From porting the code, we discovered the Lifecycle and Compose options for creating nodes. The new Lifecycle state machine available for nodes allows for better management of interactions between nodes. It allows for control in the event that a node crashes or fails to change state, affecting the behavior of other nodes. The Composite feature allows for optimized data transmission.

Furthermore, for creating AMRs, the modularity and large number of solutions for each problem allow for easy modifications to plugins and nodes, facilitating rapid implementation. The configurability of nodes enables these performance improvements with minimal code changes. Nav2's modularity allowed for quick import of code from our planner, allowing us to evaluate its advantages and disadvantages.

Finally, OpenRMF demonstrates its potential. Thanks to its tools, it allows the rapid creation of complex navigation graph paths, it also allows brand-independent robot infrastructures, allows the easy import of different robots and has an integrated package for ROS robots, allows the control of structural elements such as doors and with its dedicated task functionality it allows the control of different and customized tasks.

Chapter 16

Future work

Using OpenRMF, we can create nodes that, thanks to an extensive language model, can send tasks to the robot. We aim to implement this functionality in our warehouse, allowing us to effectively control the robot. Additionally, with the Fleet Adapter, which forms the basis of the Free Fleet package, we can manage other robots in our fleet. We've also identified ways to improve the planner.

Part VI

Appendices

Chapter 17

Simulation

All test were done on debian based machine: To see the graphical interface launched in the docker container you need to have to share the access to the X server using the command `xhost local:root`. In the code, a Dockerfile is available to build an image that is ready to use.

Bibliography

- [1] N. Abu et al. “A Comprehensive Overview of Classical and Modern Route Planning Algorithms for Self-Driving Mobile Robots”. In: *Journal of Robotics and Control (JRC)* 3 (Sept. 2022), pp. 666–678. DOI: 10.18196/jrc.v3i5.14683.
- [2] Sy-Hung Bach, Phan-Bui Khoi, and Soo-Yeong Yi. “Application of QR Code for Localization and Navigation of Indoor Mobile Robot”. In: *IEEE Access* 11 (2023), pp. 28384–28390. DOI: 10.1109/ACCESS.2023.3250253.
- [3] Luca Carlone et al. “Part1 Prelude”. In: *SLAM Handbook. From Localization and Mapping to Spatial Intelligence*. Ed. by Luca Carlone et al. Cambridge University Press.
- [4] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI*. July 2018. DOI: 10.1201/9780429489105. URL: <http://dx.doi.org/10.1201/9780429489105>.
- [5] Tully Foote. “tf: The transform library”. In: *Technologies for Practical Robot Applications (TePRA), 2013 IEEE International Conference on*. Open-Source Software workshop. 2013, pp. 1–6. DOI: 10.1109/TePRA.2013.6556373.
- [6] D. Fox, W. Burgard, and S. Thrun. “The dynamic window approach to collision avoidance”. In: *IEEE Robotics & Automation Magazine* 4.1 (1997), pp. 23–33. DOI: 10.1109/100.580977.
- [7] N.J. Gordon, D.J. Salmond, and A.F.M. Smith. “Novel approach to nonlinear/non-Gaussian Bayesian state estimation”. In: *IEE Proceedings F (Radar and Signal Processing)* 140 (2 1993), pp. 107–113. DOI: 10.1049/ip-f-2.1993.0015. eprint: <https://digital-library.theiet.org/doi/pdf/10.1049/ip-f-2.1993.0015>. URL: <https://digital-library.theiet.org/doi/abs/10.1049/ip-f-2.1993.0015>.
- [8] Wolfgang Hess et al. “Real-Time Loop Closure in 2D LIDAR SLAM”. In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1271–1278.
- [9] Steve Macenski and Ivona Jambrecic. “SLAM Toolbox: SLAM for the dynamic world”. In: *Journal of Open Source Software* 6.61 (2021), p. 2783. DOI: 10.21105/joss.02783. URL: <https://doi.org/10.21105/joss.02783>.
- [10] Steve Macenski et al. *Impact of ROS 2 Node Composition in Robotic Systems*. 2023. arXiv: 2305.09933 [cs.R0]. URL: <https://arxiv.org/abs/2305.09933>.

- [11] Steven Macenski et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [12] Steven Macenski et al. “The Marathon 2: A Navigation System”. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020.
- [13] Alexey Merzlyakov and Steven Macenski. “A Comparison of Modern General-Purpose Visual SLAM Approaches”. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2021.
- [14] T. Moore and D. Stouch. “A Generalized Extended Kalman Filter Implementation for the Robot Operating System”. In: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*. Springer, 2014.
- [15] DV Lu A. Merzlyakov M. Ferguson S. Macenski T. Moore. “From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2”. In: *Robotics and Autonomous Systems* (2023).
- [16] Kavan Singh Sikand et al. *Robofleet: Open Source Communication and Management for Fleets of Autonomous Robots*. 2021. arXiv: 2103.06993 [cs.RO]. URL: <https://arxiv.org/abs/2103.06993>.
- [17] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. The MIT Press, 2005. ISBN: 0262201623.
- [18] Victor Mayoral Vilches et al. *SROS2: Usable Cyber Security Tools for ROS 2*. 2022. arXiv: 2208.02615 [cs.CR]. URL: <https://arxiv.org/abs/2208.02615>.
- [19] Andrey Vukolov. “D-Star-Based Optimized Trajectory Planner for Mobile Robots Operating in Dense Environments”. In: *New Trends in Mechanism and Machine Science*. Ed. by Giulio Rosati, Alessandro Gasparetto, and Marco Ceccarelli. Cham: Springer Nature Switzerland, 2024, pp. 294–301. ISBN: 978-3-031-67295-8.
- [20] Andrey Vukolov. *ElettraSciComp/witmotion_IMU_QT: Version 0.9.17*. Version v0.9.17-alpha. 2022. DOI: 10.5281/zenodo.7017118. URL: <https://doi.org/10.5281/zenodo.7017118>.
- [21] Andrey Vukolov et al. “Universal Configurable Navigation and Control System for Industrial Unmanned Ground Vehicles with Differential Chassis”. In: *Advances in Mechanism and Machine Science*. Ed. by Masafumi Okada. Cham: Springer Nature Switzerland, 2023, pp. 287–301. ISBN: 978-3-031-45770-8.
- [22] Grady Williams et al. “Aggressive driving with model predictive path integral control”. In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1433–1440. DOI: 10.1109/ICRA.2016.7487277.